DRIVE247 — IMPLEMENTATION SPECIFICATION

Lead Management & Automations

Two new modules — kanban-driven lead pipeline + independent workflow engine.

Project: Drive247 (Turborepo monorepo)
Audience: Engineer + AI assistant building this feature
Status: Specification — not yet implemented
Date: 2026-05-22

Lead Management & Automations — Implementation Specification

> **Audience:** Engineer (and their AI assistant) building this feature. **Status:** Specification. Not yet implemented. **Date:** 2026-05-22 **Project:** Drive247 (Turborepo monorepo — `apps/portal`, `apps/booking`, `apps/admin`, `apps/web`) **Scope:** Two new modules — **Lead Management** (consumer of automations) and **Automations** (independent workflow engine). Both ship as opt-in tenant modules.

---

0. How to use this document

- **Read top to bottom once** — sections build on each other.
- **Reference sections by number** when implementing — every section is self-contained enough to act on, with cross-refs to dependencies.
- **Code blocks are literal** — file paths, SQL, types, function signatures, and stage names are exact and case-sensitive.
- **Reuse existing infrastructure** wherever Section 12 says so. Do not re-implement existing patterns.
- **Anti-patterns are in Section 19** — read it before starting.

Conventions

Convention	Meaning
`path/like/this.ts`	Absolute path from repo root
`snake_case`	DB column or table name
`camelCase`	TS variable / function
`PascalCase`	TS type or React component
`lead.stage_changed`	Event name in the trigger registry
MUST / **MUST NOT**	Hard requirement
SHOULD / **SHOULD NOT**	Strong default, override only with justification
MAY	Optional

---

1. Strategic Positioning

Drive247 currently owns the **rental operating system** layer — vehicles, bookings, agreements, deposits, payments, agreements, customer portal, Stripe Connect, BoldSign, Veriff, Bonzah insurance, lockbox handover.

Competitors like Rental Pal Pro own the **CRM/lead** layer (via GoHighLevel customisation), but they cannot show real vehicle availability or trigger rental operations cleanly because their pipeline lives outside their rental system.

This feature closes the gap by adding a **first-class Lead-to-Rental pipeline** inside Drive247, integrated with every existing rail. The Automations module makes the same pipeline scriptable by non-engineers.

One-line positioning

> Rental Pal Pro is a CRM that knows about rentals. Drive247 is a rental operating system with a CRM built in.

Non-goals

- **MUST NOT** implement any insurance-document tampering workflow (the "PDF editor" feature in Rental Pal Pro). See Section 19.
- **MUST NOT** build a generic chatbot. The AI layer is for structured extraction, ranking, and action suggestion — not conversation.

---

2. Locked Decisions

#	Decision	Choice
1	Lead data model	New `leads` table. `customers` row created only on conversion. Existing `enquiries` rows migrated in with `source='legacy_enquiry'`.
2	V1 scope	Kanban board + Apply funnel + 3-column workspace + matching engine + offer link + manual stage moves + convert-to-rental. Hardcoded automations (welcome SMS, stale-lead reminder, lost-after-48h) inside the lead engine, **extracted into the Automations module in V2**.
3	Application form	Fixed schema (Section 6.2). Operator-configurable form builder is V3.
4	Tenant scope	Opt-in. New tenant flags `lead_management_enabled` (default `false`) and `automations_enabled` (default `false`).

---

3. Glossary

Term	Meaning
Lead	A pre-rental opportunity. Has a stage and a state machine. Lives in `public.leads`.
Application	A multi-step form on the booking site that creates a lead with `source='application'`.
Quick Enquiry	The existing short contact-form path (`source='quick_enquiry'`). Creates a lead with less data.
Stage	The lead's current pipeline position (`new`, `approved`, `waitlist`, etc.). See Section 6.1.
Score	`lead_score INT (0–100)` + `score_band` (`hot` / `warm` / `cold` / `risk`). Computed by the scoring engine.
Conversation	A persistent message thread keyed by `lead_id` and/or `customer_id`. Survives lead → customer conversion.
Offer link	A unique short-URL that shows a curated set of vehicles to a lead, with one-click selection. Lives in `public.lead_offers`.
Automation	A user-built workflow: trigger + ordered steps. Lives in `public.automations`.
Run	A single execution of an automation against a specific entity. Lives in `public.automation_runs`.
Trigger registry	Code-defined enumeration of events that can fire automations (Section 7.1).
Matching engine	Function returning ranked vehicle options for a given lead's request (Section 6.5).
Tenant	A rental operator on the Drive247 platform (existing concept).

---

4. User Journeys

4.1 Customer journey (Marcus)

1. Marcus sees a Facebook ad for a rental tenant. Clicks → lands on `tenant.drive-247.com`.
2. Two CTAs: **Book a Vehicle** (existing booking flow) and **Apply for a Rental** (new, this spec).
3. Marcus clicks Apply. Multi-step form (7 steps, Section 6.2). Submits.
4. Confirmation page: *"We've got your application. Watch your phone."*
5. Instant SMS from tenant: *"Hi Marcus, we received your application. Reply here if you have any questions."*
6. ~15 minutes later, SMS from admin: *"Hey Marcus — quick question, are you available Friday for pickup?"*
7. Conversation continues via SMS. Marcus uploads licence via a secure link the admin sends him.
8. Admin sends an **offer link** (Section 6.6) with 3 curated vehicles + Marcus's dates.
9. Marcus opens the link on his phone, picks Civic, confirms dates.
0. SMS from system: *"Awesome — here's your agreement and deposit link."*
1. Marcus signs (BoldSign, existing flow) and pays deposit (Stripe Connect, existing flow).
2. SMS from system: pickup scheduler link.
3. Marcus picks a pickup time. 24h + 2h reminders fire.
4. Marcus picks up the keys. Lead is now a customer with an active rental.

4.2 Operator journey (admin Sarah at the tenant)

1. New card appears in **Lead Hub** kanban → **New Lead** column. Realtime push, optional desktop notification.
2. Sarah clicks the card → 3-column **Lead Workspace** opens (full page, Section 6.4).
3. Left column shows Marcus's application data + lead score = **Hot**.
4. Middle column is empty (no messages yet) + a **template picker** at the composer.
5. Right column shows the matching engine has already run (Section 6.5) and lists 3 ranked alternatives + Sarah's actual requested Corolla (partially available).
6. Sarah taps **Request Documents** in the right column → SMS goes to Marcus with a secure upload link.
7. Marcus uploads. Veriff/AI verification runs (existing flow). Stage auto-moves to **Docs Verified**.
8. Sarah reviews + taps **Approve**.
9. Sarah selects 3 vehicles in the matching engine → taps **Create offer link** → reviews customisation panel → sends via SMS.
0. Marcus selects Civic. Stage auto-moves to **Offer Accepted**. System suggests next action: send agreement.
1. Sarah taps **Send Agreement** → BoldSign flow fires. Marcus signs. Stage moves to **Agreement Signed**.
2. Stripe deposit link fires. Marcus pays. Stage moves to **Deposit Paid**.
3. Sarah taps **Convert to Rental** → conversion flow runs (Section 6.9). Lead card greys out. Rental appears in active rentals list.

4.3 Lead → Customer journey

- A lead's `conversation` row has `lead_id` set, `customer_id = null`.
- On conversion, the conversion handler creates the `customers` row, then UPDATES the conversation: `customer_id = <new>`. **Does not** create a new conversation.
- All future messages between Marcus and the tenant — whether sent from Lead Hub, Customer Messages, or Customer Profile — read and write the same conversation row.
- After conversion, the lead card stays visible in Lead Hub in a `converted` terminal stage for 30 days, then archived.

---

5. System Architecture

6. Lead Management

6.1 Stage State Machine

Stages

Stage	Description	Terminal?
`new`	Just submitted, untouched	No
`contacted`	Admin reached out	No
`docs_requested`	Docs link sent	No
`docs_submitted`	Lead uploaded ≥1 doc	No
`docs_verified`	All required docs verified (Veriff/AI passes)	No
`docs_failed`	Verification failed; needs operator decision	No
`approved`	Operator approved the lead	No
`vehicle_offered`	Offer link sent	No
`offer_accepted`	Lead picked a vehicle from offer	No
`agreement_sent`	BoldSign agreement dispatched	No
`agreement_signed`	Lead signed	No
`deposit_paid`	Deposit collected	No
`pickup_scheduled`	Pickup slot booked	No
`converted`	Active rental exists (terminal positive)	**Yes**
`waitlist`	Parked due to no vehicle	No
`lost`	No response or declined (terminal negative)	**Yes**
`blacklisted`	Flagged (terminal negative)	**Yes**

Allowed transitions

```
new → contacted, docs_requested, waitlist, lost, blacklisted
contacted → docs_requested, approved, waitlist, lost, blacklisted
docs_requested → docs_submitted, docs_failed, lost, blacklisted
docs_submitted → docs_verified, docs_failed
docs_verified → approved, lost
docs_failed → docs_requested, approved (operator override), lost, blacklisted
approved → vehicle_offered, waitlist, lost
vehicle_offered → offer_accepted, lost  (auto-expires after offer's expires_at → lost)
offer_accepted → agreement_sent, lost
agreement_sent → agreement_signed, lost
agreement_signed → deposit_paid, lost
deposit_paid → pickup_scheduled, lost
pickup_scheduled → converted, lost
waitlist → approved, vehicle_offered, lost, blacklisted
lost → new (operator can resurrect)
blacklisted → new (operator can unblacklist)
```

The state machine **MUST** be enforced in:

- TS lib: `apps/portal/src/lib/lead-stage-machine.ts` (`canTransition(from, to): boolean`).
- DB trigger: `validate_lead_stage_transition()` on UPDATE of `leads.stage`.

Auto transitions (system-driven)

Trigger	From	To
Lead uploaded last required doc	`docs_requested`	`docs_submitted`
Veriff/AI/CMD verification passes	`docs_submitted`	`docs_verified`
Veriff/AI/CMD verification fails	`docs_submitted`	`docs_failed`
Offer link accepted by lead	`vehicle_offered`	`offer_accepted`
Offer link expires unaccepted	`vehicle_offered`	`lost` (Phase 2: configurable to `waitlist` instead)
BoldSign webhook: agreement signed	`agreement_sent`	`agreement_signed`
Stripe webhook: deposit captured	`agreement_signed`	`deposit_paid`
Pickup slot scheduled	`deposit_paid`	`pickup_scheduled`
Rental key handover completed	`pickup_scheduled`	`converted`
48h no activity from `new` / `contacted` / `docs_requested`	(above)	`lost` (configurable per tenant)

6.2 Customer Apply Flow

Route

`apps/booking/src/app/apply/page.tsx` — multi-step wizard. Each step is one screen. Progress bar at top showing 7 steps.

`apps/booking/src/app/apply/submitted/page.tsx` — confirmation screen.

Step schema (fixed, V1)

Use **React Hook Form + Zod**. Define one master schema and per-step partial schemas in `apps/booking/src/client-schemas/apply.ts`.

```
// apps/booking/src/client-schemas/apply.ts
import { z } from "zod";

export const applySchema = z.object({
  // Step 1 — About you
  fullName: z.string().trim().min(2).max(100),
  dateOfBirth: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  email: z.string().email().max(255),
  phone: z.string().refine(/* 7–15 digits, same as enquirySchema */),
  addressLine1: z.string().min(2).max(200),
  addressLine2: z.string().max(200).optional(),
  city: z.string().min(2).max(100),
  state: z.string().min(2).max(100),
  postalCode: z.string().min(3).max(20),
  country: z.string().length(2).default("US"),

  // Step 2 — Driver
  licenceNumber: z.string().min(3).max(50),
  licenceState: z.string().min(2).max(100),
  licenceExpiry: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  yearsDriving: z.coerce.number().int().min(0).max(80),
  hasViolations: z.boolean(),
  violationsDescription: z.string().max(2000).optional(),

  // Step 3 — Rental intent
  purpose: z.enum(["uber", "lyft", "doordash", "instacart", "personal", "delivery", "other"]),
  rideshare Platforms: z.array(z.string()).max(10).default([]),
  neededByDate: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  rentalLengthTarget: z.enum(["daily", "weekly", "monthly"]),
  vehicleInterestType: z.enum(["specific", "class", "any"]),
  vehicleId: z.string().uuid().optional(),     // when 'specific'
  vehicleClass: z.string().optional(),         // when 'class' (e.g. 'sedan', 'suv')
  startDate: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  endDate: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),

  // Step 4 — Financial
  canPayDeposit: z.boolean(),
  depositComfortAmount: z.coerce.number().int().min(0).optional(),
  weeklyBudget: z.coerce.number().int().min(0).optional(),

  // Step 5 — History
  rentedBefore: z.boolean(),
  rentedFromUsBefore: z.boolean(),
  rideshareAccountActive: z.boolean(),
  rideshareTier: z.string().max(100).optional(),

  // Step 6 — Documents (paths to uploaded storage objects)
  licencePhotoUrl: z.string().url().optional(),
  selfieUrl: z.string().url().optional(),
  rideshareProofUrl: z.string().url().optional(),

  // Step 7 — Review
  termsAccepted: z.literal(true),
  marketingConsent: z.boolean().default(false),

  // Honeypot
  hpField: z.string().optional(),
}).refine((d) => Date.parse(d.endDate) >= Date.parse(d.startDate), {
  path: ["endDate"],
  message: "End date must be on or after start date",
});

export type ApplyFormValues = z.infer<typeof applySchema>;
```

Submission flow

1. Client validates each step before allowing **Next**.
2. Final submit POSTs to `submit-application` edge function with full payload.
3. Edge function:
    1. Validates payload (same Zod schema, server-side via Deno Zod).
    2. Inserts `leads` row with `source='application'`, `application_data=<payload minus identity fields>`, `stage='new'`.
    3. Inserts `lead_documents` rows for any uploaded files.
    4. Runs `check-blacklist-match` synchronously. If hard match → sets `stage='blacklisted'` and `blacklist_match_id=<id>`.
    5. Otherwise: runs `compute-lead-score`, sets `lead_score` and `score_band`.
    6. Inserts `conversations` row with `lead_id=<new>`.
    7. Emits event `lead.created` to `automation_event_queue` (and `lead.application_submitted`).
    8. Sends acknowledgement SMS to lead via `send-lead-message` (system-triggered; hardcoded template in V1).
    9. Returns `{ leadId, status: 'received' }` to client.
4. Client redirects to `/apply/submitted` with status banner.

Document upload

- Files **MUST** be uploaded to a new storage bucket `lead-documents` (10MB max per file, JPG/PNG/PDF only).
- Bucket creation:

```
INSERT INTO storage.buckets (id, name, public)
VALUES ('lead-documents', 'lead-documents', false);
```

- Storage policies: same pattern as `gig-driver-images` (Section 12).
- Upload happens **client-side before final submission**, using a presigned URL from a new edge function `lead-document-presign` (or reuse existing pattern — confirm with `gig-driver-images.ts` upload pattern).

Honeypot + rate limiting

- `hpField` honeypot — if non-empty, server returns 200 with `status: 'received'` but inserts nothing.
- Rate limit by IP via `ip_address` column + idx_leads_ip_recent (mirror enquiries pattern). 5 per IP per hour.

6.3 Portal Kanban Board

Route

`apps/portal/src/app/(dashboard)/leads/page.tsx`

Columns (left-to-right)

1. New
2. Contacted
3. Docs Requested
4. Docs Submitted / Verified (merged column, badge differentiates)
5. Approved
6. Vehicle Offered
7. Offer Accepted
8. Agreement Sent / Signed (merged)

9. Deposit Paid / Pickup Scheduled (merged)
0. Waitlist (separate panel — not in main flow)
1. Lost
2. Blacklisted

> Reasoning: 8 visible columns + Waitlist/Lost/Blacklisted accessible via tabs at the top of the board. **MUST** not display 16 columns.

Board UI

- **Top bar**: search (name/phone/email), filter chips (assigned-to-me, score band, source, vehicle interest, date range), tenant-scoped count, "+ New Lead" button (manual creation).
- **Tabs**: `Active` (default — shows columns 1–9), `Waitlist`, `Lost`, `Blacklisted`.
- **Cards**: rendered by `LeadCard` component. Show:
  - Name + score chip (🔥/Warm/Cold/⚠)
  - Phone (last 4) + source icon
  - Requested vehicle (class or name)
  - Date range
  - Time in stage
  - Assigned avatar
  - Unread message badge
  - Stale indicator (orange dot if no activity > tenant's stale threshold)
- **Drag-drop**: `@dnd-kit/core` (already common in similar React stacks). Dragging a card to a new column calls the stage-transition API (`PATCH /leads/:id/stage`). Optimistic UI with rollback on error.
- **Drag-drop respects state machine** — invalid drops are visually rejected with a toast.
- **Click card → routes to `/leads/[id]`** (full-page workspace, Section 6.4). **MUST NOT** open a drawer.
- **Realtime**: Supabase realtime subscription on `leads` table (filtered by `tenant_id`) updates card positions live.

Empty state

- If `lead_management_enabled = false`: show a setup CTA "Enable Lead Management" → routes to `/settings/lead-management` (Section 14).
- If enabled but no leads: show empty-state illustration + "Get your first lead — share your apply link: `tenant.drive-247.com/apply`".

6.4 3-Column Lead Workspace

Route

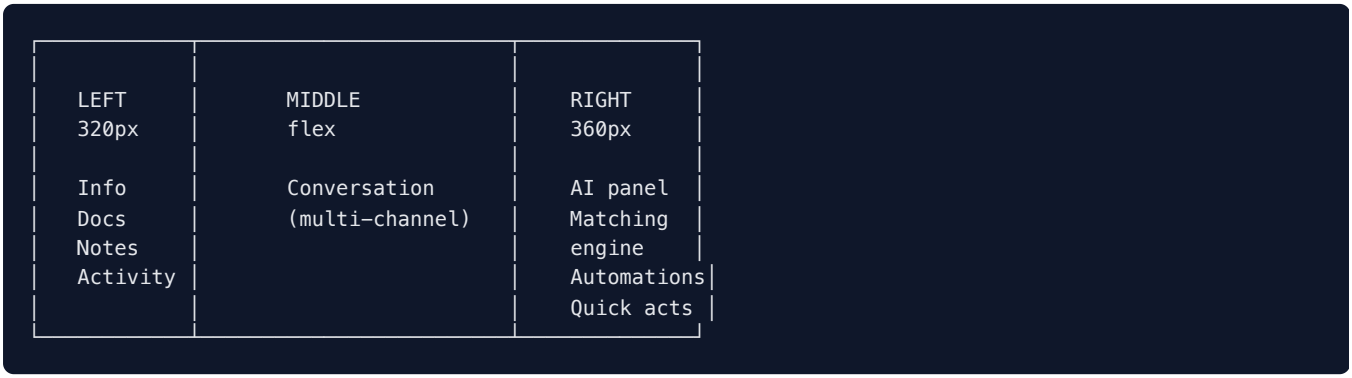
`apps/portal/src/app/(dashboard)/leads/[id]/page.tsx` — full page. Three columns + top action bar.

Top action bar

- Breadcrumb: Leads / [Lead Name]
- Stage selector (constrained dropdown — only valid transitions)
- Score chip
- Assigned-to selector
- Action buttons: **Convert to Rental** (only enabled when `stage IN ('deposit_paid','pickup_scheduled')`), **More ▾** (Add to Blacklist, Mark Lost, Archive, Delete)

- Realtime presence dots (other staff currently viewing this lead)

Layout



Below 1400px width: collapse to tabs **Info | Chat | AI**. Default tab = Chat.

Left column — Lead Info Panel

Component: `apps/portal/src/components/leads/lead-info-panel.tsx`

Sections (top to bottom):

1. **Header**
   - Avatar (initials), name, score chip, stage badge
   - Quick action icons: 📞 Call · 💬 SMS · ✉ Email · 🟢 WhatsApp (each focuses the composer in middle column)
2. **Application summary** — pretty-rendered, read-only
   - Purpose, rideshare app + tier
   - Vehicle interest, date range, rental length
   - Deposit readiness, weekly budget
   - Years driving, violations summary
   - Address, DOB
3. **Documents**
   - Per document: chip with status (`pending` / `uploaded` / `verifying` / `verified` / `failed` / `expired`)
   - Click → modal preview
   - Action buttons per doc: Request again, Re-verify, Mark expired, Delete
   - Reuses Veriff/AI/CMD identity verification flows (see Section 12 reuse map)
4. **Tags**
   - Operator-set free-text tags (`vip` , `returning` , `out-of-state` , `gig-uber`)
   - Tags can be used as automation filters
5. **Notes**
   - Pinned notes at top
   - Note thread, newest at top
   - Add-note inline composer
   - Internal only — never sent to lead
6. **Activity timeline**
   - Read-only chronological feed from `lead_activity` table
   - Event types: `stage_changed` , `message_sent` , `doc_uploaded` , `doc_verified` , `offer_sent` , `offer_opened` , `offer_accepted` , `automation_started` , `automation_completed` , `score_changed` , `assigned` , `note_added`

- Each row: timestamp, actor type + name, human-readable event description

Middle column — Communication Panel

Component: `apps/portal/src/components/leads/lead-communication-panel.tsx`

This is the unified multi-channel inbox for this lead.

Visual

- Bubble layout, like iMessage/WhatsApp/GHL conversations.
- **Lead messages** (inbound): left-aligned, grey background.
- **Outbound messages**: right-aligned, channel-coloured background.
- **Internal notes**: yellow, centred (never sent).
- **System events**: centred grey pill (`Stage changed to Approved by Sarah`).
- **AI suggestions**: small dismissible chip above composer when present.

Per-message decorations

Channel	Icon	Bubble colour	Notes
SMS	📱	green	Sent via Twilio. Show `delivered` / `read` / `failed` ticks.
Email	✉	blue	Show subject + truncated body, click to expand
WhatsApp	🟢	WhatsApp green	Sent via Twilio; supports content templates per Drive247's existing WhatsApp infra
Internal Note	📝	yellow	Staff-only
System	⚙	grey pill	Stage changes, automations, etc.
Call Summary	📞	grey	AI transcript of inbound/outbound calls (Phase 2 hookup to existing Call Recording feature)

Composer

- **Channel toggle**: SMS / Email / WhatsApp / Note (tabs above composer)
- **Template picker**: dropdown of templates from `lead_message_templates` table (per-tenant, with `{{variables}}`). System provides default templates if tenant has none configured.
- **Variable insert**: type `{{` to autocomplete variables (`{{first_name}}`, `{{vehicle}}`, `{{start_date}}`, `{{offer_link}}`, `{{agreement_link}}`, `{{deposit_link}}`, `{{pickup_link}}`, `{{tenant_name}}`).
- **Attachments**:
  - Doc upload link (generates short-lived upload URL for lead)
  - Payment link (generates Stripe preauth/checkout link)
  - Agreement link (triggers BoldSign send)
  - Pickup scheduler link
  - File attachment (image/pdf — only for WhatsApp/Email)
- **Send button** + keyboard shortcut (Cmd/Ctrl + Enter)
- **AI suggestion chip** (when present): one-tap action, e.g. *"Marcus hasn't replied in 18h. Send follow-up?"* → fills composer with drafted message.

Inbound message handling

- **Inbound SMS**: Twilio webhook → `inbound-sms-webhook` edge function. Resolves the lead by phone (normalised). Appends to conversation. Broadcasts via realtime.
- **Inbound Email**: SES/Resend webhook → `inbound-email-webhook`. Resolves lead by email or by `In-Reply-To` header containing a conversation ID.
- **Inbound WhatsApp**: Twilio WhatsApp webhook → `inbound-sms-webhook` (same function, channel branch on `whatsapp:` prefix).
- **No-match**: if no lead matches, create a new lead with `source='inbound_sms'` (or `inbound_email` / `inbound_whatsapp`), `stage='new'`.

Realtime

- Channel name: `tenant_${tenantId}_conversation_${conversationId}`.
- Subscribe in `useConversationMessages(conversationId)` hook.
- Mirrors existing `RealtimeChatContext` pattern.

Right column — AI / Matching / Automations Panel

Component: `apps/portal/src/components/leads/lead-ai-panel.tsx`

Three stacked sections, all collapsible.

Section 1 — AI Next Action

Single prominent card showing the most useful suggested action right now.

Examples by stage:

- `new` → *"Send welcome message + ask for documents."* (one-tap)
- `docs_submitted` → *"All docs uploaded. Run Veriff?"*
- `docs_verified` → *"Marcus passed verification. Approve?"*
- `approved` → *"Marcus matches Civic 84% likely. Send offer link?"*
- `vehicle_offered` (opened, no action) → *"Marcus opened the offer 2h ago. Send follow-up?"*
- `agreement_sent` → *"Agreement unsigned for 12h. Send reminder?"*
- `lost` → empty (no suggestion).

Implemented by `ai-suggest-next-action` edge function (Section 11.2).

Section 2 — Matching Engine

Sub-component: `apps/portal/src/components/leads/lead-matching-engine.tsx`

See Section 6.5 for full spec.

Section 3 — Automations & Quick Actions

Sub-component: `apps/portal/src/components/leads/lead-automations-panel.tsx`

- **Active automations on this lead** — list of running `automation_runs` with status, current step, resume_at.
- **Pause / resume per run** — toggles `automation_runs.status` between `running` / `paused`.
- **Stage SLA** — *"In Approved for 1d 4h, SLA 24h ❗ overdue"* (configurable per tenant in Phase 2).
- **Attach automation** dropdown — lists published automations. On select:
  - If automation's `trigger_type='manual'`: "Run now" button.
  - If event-driven: "Already runs on `{trigger}` events" (informational).
- **Quick action buttons** (call existing edge functions):

- Request Documents
- Run Veriff (existing `create-veriff-session`)
- Check Bonzah eligibility (existing `bonzah-create-quote`)
- Send Agreement (existing BoldSign send)
- Send Payment Link (existing `create-checkout-session` / `create-preauth-checkout`)
- Schedule Pickup
- Convert to Rental (only when stage allows)
- Mark as Lost
- Add to Blacklist

6.5 Matching Engine

Purpose

Given a lead's request (vehicle preference, dates, rental type), return a ranked list of matched vehicles with metadata. Used by:

- The right column of the lead workspace.
- The offer-link builder.
- Automations (Phase 2 action: `recommend_vehicle`).

Inputs

```
type MatchInput = {
  leadId: string;
  // Or, when running synthetically:
  tenantId: string;
  vehicleInterest: { type: "specific"; vehicleId: string } | { type: "class"; class: string } | { type: "an
  startDate: string;          // YYYY-MM-DD
  endDate: string;            // YYYY-MM-DD
  rentalType: "daily" | "weekly" | "monthly";
  purpose?: string;           // uber, lyft, …
  weeklyBudget?: number;
  depositComfortAmount?: number;
};
```

Outputs

```
type MatchResult = {
  generatedAt: string;
  options: MatchOption[];
};

type MatchOption = {
  kind: "single" | "stitched" | "conditional";
  vehicles: Array<{
    vehicleId: string;
    name: string;       // make + model
    class: string;
    photoUrl: string | null;
    startDate: string;
    endDate: string;
    weeklyRate: number;
    dailyRate: number;
    available: "full" | "partial" | "unavailable";
  }>;
  conditions?: string[];     // e.g. ["Higher deposit ($500)", "Start 27 May instead of 25 May"]
  matchScore: number;        // 0–100 deterministic
  aiScore?: number;          // 0–100 from ai-rank-matches (added when AI present)
  acceptanceProbability?: number; // 0–1 from AI
  reasoning?: string[];      // why this is offered
  totalPrice: number;        // for the period
  budgetFit: "under" | "within" | "over";
  insuranceEligible: boolean;
};
```

Algorithm (deterministic core)

1. **Resolve candidate vehicles**:
   - If `vehicleInterest.type === 'specific'`: the requested vehicle + class-siblings.
   - Else: all tenant vehicles in matching class (or all if `any`).
2. **Filter** by eligibility:
   - Active vehicle (`vehicles.is_active = true`).
   - Rental type supported (`available_daily` / `available_weekly` / `available_monthly` flag).
   - Min rental hours satisfied (tenant setting + vehicle override).
   - Rideshare-approved if `purpose` is gig.
3. **Compute availability** per vehicle:
   - Pull bookings overlapping the date range (existing booking calendar logic).
   - Pull `external_bookings` (Turo/Airbnb iCal).
   - Pull maintenance windows.
   - Pull tenant buffer time between rentals.
   - Result: `full` / `partial` (with sub-windows) / `unavailable`.
4. **Price** each option using existing pricing engine (daily/weekly/monthly tiers + dynamic pricing surcharges + min hours).
5. **Compute matchScore**:
   - Closeness to original vehicle (100 if same, 80 if same class, 60 if adjacent class).
   - Date coverage (% of requested days available).
   - Price fit (penalise if outside `weeklyBudget`).
   - Utilisation priority (boost less-rented vehicles — configurable).

6. **Detect stitched options**: if no single vehicle covers full period, search 2-vehicle combinations that do.
7. **Detect conditional options**: vehicles that are unavailable but become available with one of:
   - +1/+2 day start shift
   - -1/-2 day end shift
   - Higher deposit (if vehicle has `higher_deposit_unlocks_renters = true`)
8. **Sort by matchScore DESC**, return top 8.

AI rerank layer

After deterministic results, call `ai-rank-matches` edge function (Section 11.2). It returns:

- `aiScore` per option
- `acceptanceProbability` per option
- `reasoning` per option (1-2 sentences)

Final ordering = combination of `matchScore` (deterministic, 60%) + `aiScore` (40%).

If AI rerank fails or is disabled, fall back to deterministic order with no aiScore.

Implementation

- Edge function: `supabase/functions/run-matching-engine/index.ts`
- Pure TS helper extracted into shared lib: `supabase/functions/_shared/matching.ts` (so it can be reused by `create-offer-link` and automation actions).
- Portal hook: `apps/portal/src/hooks/use-matching-engine.ts` — React Query keyed by `['matching', leadId, lastUpdated]`.

6.6 Customisable Offer Link

Concept

Admin curates 1–N vehicles + dates + pricing → generates a short URL → sends via SMS/email/WhatsApp → lead opens on mobile → picks vehicle + confirms dates → lead auto-advances.

Data model

See Section 8.1 for full `lead_offers` table SQL.

Short code generation: 8-char random base62 (`nanoid` style). Unique constraint on `short_code`.

Offer-builder UI (right column → "Create offer link" button)

Component: `apps/portal/src/components/leads/offer-builder-dialog.tsx`

Form:

1. **Vehicles** (selected from matching engine, removable, reorderable)
   - Per-vehicle: price override input, date override input
2. **Default dates** (pre-filled from lead's requested dates)
3. **Date flexibility**: ±0 / ±1 / ±2 / ±3 / ±7 days (lead can shift dates within this window)
4. **Deposit**: amount in dollars
5. **Custom message** (textarea) — top of offer page. Pre-filled by `ai-draft-message`.
6. **Expiry**: 12h / 24h / 3d / 7d (radio)
7. **Show prices**: toggle (some tenants prefer to quote in chat)

8. **Send method**: SMS / Email / WhatsApp / Copy Link

On submit:

1. POST to `create-offer-link`.
2. Edge function:
   1. Inserts `lead_offers` row with `status='pending'`.
   2. Generates short URL: `https://{tenant_slug}.drive-247.com/offer/{short_code}`.
   3. If `sendMethod` ≠ `copy`: calls `send-lead-message` to dispatch the message with `{{offer_link}}` injected.
   4. Records `lead_activity` event `offer_sent`.
   5. Transitions lead to `vehicle_offered`.
   6. Emits `lead.offer_sent` event.
3. Returns `{ offerId, shortCode, url }`.

Customer offer page

Route: `apps/booking/src/app/offer/[code]/page.tsx`

- Server-rendered, no auth required.
- Resolves offer by `short_code` (must be unique, not expired).
- If expired → renders "This offer has expired" page; auto-emits `lead.offer_expired` event.
- If valid → renders mobile-first page (Section 6.6 customer UI mockup, see chat history).
- Tracks views: `view_count++`, `first_viewed_at` if null, `last_viewed_at` always. Emits `lead.offer_opened` on first view.
- Customer picks a vehicle → POST to `accept-offer`.

`accept-offer` edge function

1. Validates `short_code` + not expired + `status IN ('pending','viewed')`.
2. Validates picked vehicle is in `vehicles` array.
3. Validates dates are within `default_start_date ± date_flex_days` window.
4. Re-checks vehicle is still available for those dates (concurrency guard — could have been claimed elsewhere).
5. Updates `lead_offers`: `status='accepted'`, `accepted_vehicle_id`, `accepted_start_date`, `accepted_end_date`, `accepted_at`.
6. Transitions lead to `offer_accepted`.
7. Emits `lead.offer_accepted` event.
8. Returns `{ status: 'accepted' }` for the customer page to render success state.

Concurrency

- Vehicle availability is re-checked at acceptance time. If unavailable, return 409 with `reason: 'vehicle_just_taken'` and render the page with the other options remaining + apology.
- **MUST NOT** lock vehicles when offers are created (would block other operators). Only on acceptance.

6.7 Blacklist Engine

Data model

See Section 8.1 for `blacklist_entries` SQL.

Match logic

Implemented in `check-blacklist-match` edge function.

Inputs: `{ tenantId, phone, email, licenceNumber, fullName }`

Algorithm:

1. **Normalise**:
   - Phone → digits only, prepend country code (default `+1` US, fallback per tenant setting).
   - Email → lowercase, trim.
   - Licence → strip whitespace + uppercase.
   - Name → lowercase, strip punctuation.
2. **Exact-match** against `blacklist_entries`:
   - Hard match: phone OR email OR licence equal.
   - Soft match: same tenant, name similar (Levenshtein ≤ 2) AND any of (phone or email partial).
3. **Cross-tenant** (Phase 2): Look across all tenants if `tenants.cross_tenant_blacklist_enabled = true`. Returns matches with origin tenant masked.
4. Returns `{ matchType: 'none' | 'hard' | 'soft', entries: [...] }`.

Effect on lead

- **Hard match**: lead created with `stage='blacklisted'`, `blacklist_match_id=<id>`. No SMS sent. Card lands in Blacklisted tab with red flag.
- **Soft match**: lead created in normal stage, but right-column AI Next Action shows *"Possible blacklist match (soft). Review?"* — operator decides.

Admin actions

- **Add to blacklist** from a lead card: opens dialog asking reason; creates `blacklist_entries` row; sets lead to `blacklisted`; sends polite decline SMS (template, NEVER mentioning "blacklist").
- **Remove from blacklist**: deletes the matching row. Lead can re-apply.

6.8 Dedup & Merge

When does dedup run?

Every time a lead is submitted or an inbound message creates a candidate lead.

Algorithm

1. Normalise incoming `phone` and `email`.
2. Search `leads` table for same tenant + same `phone_normalised` or same `email_lower`.
3. Search `customers` table for the same.
4. Output: `{ matchType: 'none' | 'existing_lead' | 'existing_customer', matches: [...] }`.

Behaviour

- **Existing active lead** (any stage except `lost` / `blacklisted` / `converted`):
  - **MUST NOT** create new lead.
  - Append new submission to existing lead's `application_data` history (`application_data.submissions[]`).
  - Append a system event in the conversation: *"Submitted another application on {date}"*.
  - Bring existing lead to the top of the board.

- **Existing customer** (already converted):
  - Create new lead with `customer_id` pre-set (linking).
  - Existing conversation continues; new system event added.
- **Existing lost/blacklisted lead**:
  - If lost: resurrect to `new`, append submission.
  - If blacklisted: create new lead in `blacklisted` stage, link to source blacklist entry.

Implementation

- Edge function `check-lead-duplicate` (or co-located inside `submit-application`).
- Stored normalisation columns on `leads`: `phone_normalised`, `email_lower` (filled by DB trigger).
- Indexes on those columns.

6.9 Lead → Customer Conversion

Convert action enabled when `stage IN ('deposit_paid', 'pickup_scheduled')`.

Conversion handler

Edge function: `convert-lead-to-rental`

Steps (single transaction):

1. Validate lead is in convertible stage.
2. Create `customers` row from lead's identity fields + `application_data`.
3. Create `customer_users` link if a corresponding `auth.users` row exists (or queue invite email).
4. Update `leads.customer_id = <new>`, `leads.converted_at = NOW()`.
5. Update `conversations.customer_id = <new>` (lead_id stays set).
6. Create `rentals` row using:
   - `accepted_vehicle_id` from `lead_offers` (or operator-chosen vehicle)
   - `accepted_start_date` / `accepted_end_date`
   - Pricing from matching engine snapshot
   - Existing agreement reference (if BoldSign already signed)
   - Existing deposit reference (if Stripe already captured)
7. Set lead `stage='converted'`, `converted_to_rental_id=<new>`.
8. Emit events: `lead.converted`, `rental.created`.
9. Insert `lead_activity` entry.
0. Return `{ customerId, rentalId }`.

What stays alive after conversion

- `conversations` row: both `lead_id` and `customer_id` set.
- All `conversation_messages` stay readable from both Lead Hub and Customer Messages.
- Lead card visible in Kanban "Converted" pseudo-column (or Archived tab) for 30 days then hidden by default filter.

---

7. Automations Module

7.1 Trigger Registry

Defined in `apps/portal/src/lib/automation-event-registry.ts` AND `supabase/functions/_shared/automation-events.ts`. **MUST be kept in sync** (both files import a generated registry, or one re-exports the other — use a JSON-as-truth file `automation-events.json` if needed).

V1 events (lead-only)

Event name	Entity	Trigger condition	Payload variables
`lead.created`	lead	New row inserted into `leads`	`lead_id, source, score, score_band, vehicle_class, start_date, end_date`
`lead.application_submitted`	lead	`source='application'` insert	(same as above) + `application_data`
`lead.stage_changed`	lead	`leads.stage` UPDATE	`lead_id, from_stage, to_stage, actor_id, actor_type`
`lead.docs_requested`	lead	Stage moves to `docs_requested`	`lead_id, requested_docs`
`lead.docs_submitted`	lead	First doc upload	`lead_id, doc_types`
`lead.docs_verified`	lead	All docs verified	`lead_id`
`lead.docs_failed`	lead	Any doc verification failed	`lead_id, failure_reason`
`lead.score_changed`	lead	`lead_score` UPDATE crosses band	`lead_id, from_band, to_band, score`
`lead.assigned`	lead	`assigned_to` UPDATE	`lead_id, from_user_id, to_user_id`
`lead.stale_24h`	lead	No activity for 24h (cron-emitted)	`lead_id, last_activity_at`
`lead.stale_48h`	lead	No activity for 48h (cron-emitted)	`lead_id, last_activity_at`
`lead.lost`	lead	Stage → `lost`	`lead_id, reason`
`lead.blacklisted`	lead	Stage → `blacklisted`	`lead_id, reason`
`lead.converted`	lead	Stage → `converted`	`lead_id, customer_id, rental_id`
`lead.offer_sent`	lead	Offer link created and sent	`lead_id, offer_id, vehicles`
`lead.offer_opened`	lead	First view of offer page	`lead_id, offer_id`
`lead.offer_accepted`	lead	Lead picked from offer	`lead_id, offer_id, vehicle_id, dates`
`lead.offer_expired`	lead	Offer expired without acceptance	`lead_id, offer_id`
`lead.inbound_message`	lead	Inbound SMS/email/WhatsApp received	`lead_id, channel, body`
`manual`	any	Operator clicks "Run now"	`entity_type, entity_id, started_by`

Trigger config (filters)

Each automation may filter on payload fields. Example: `lead.stage_changed` with `{ to_stage: 'approved' }` only fires on transitions to Approved.

Stored as `automations.trigger_config JSONB`. Builder UI generates this from form inputs.

Future event sources (V3+)

`rental.*`, `payment.*`, `booking.*`, `customer.*` — registry is designed to expand. New events plug in without changes to the engine.

7.2 Action Types

V1 actions:

Action	Config	Behaviour
`sms`	`{ template_id?, body, channel_from? }`	Send SMS via Twilio. `body` supports `{{variables}}`.
`email`	`{ template_id?, subject, body, from_address? }`	Send email via SES/Resend.
`wait`	`{ duration: { value, unit } }` (`unit: minutes	hours
`condition`	`{ expression }`	Evaluate an expression against payload + entity. Branch to `branch='true'` or `branch='false'` children. Expressions: `lead.score_band == 'hot'`, `lead.purpose == 'uber'`, `payload.from_stage == 'new'`. Use a safe expression evaluator (whitelist of operators and fields).
`stop`	`{}`	End the run with `status='completed'`.

Phase 2 actions (do not implement in V1):

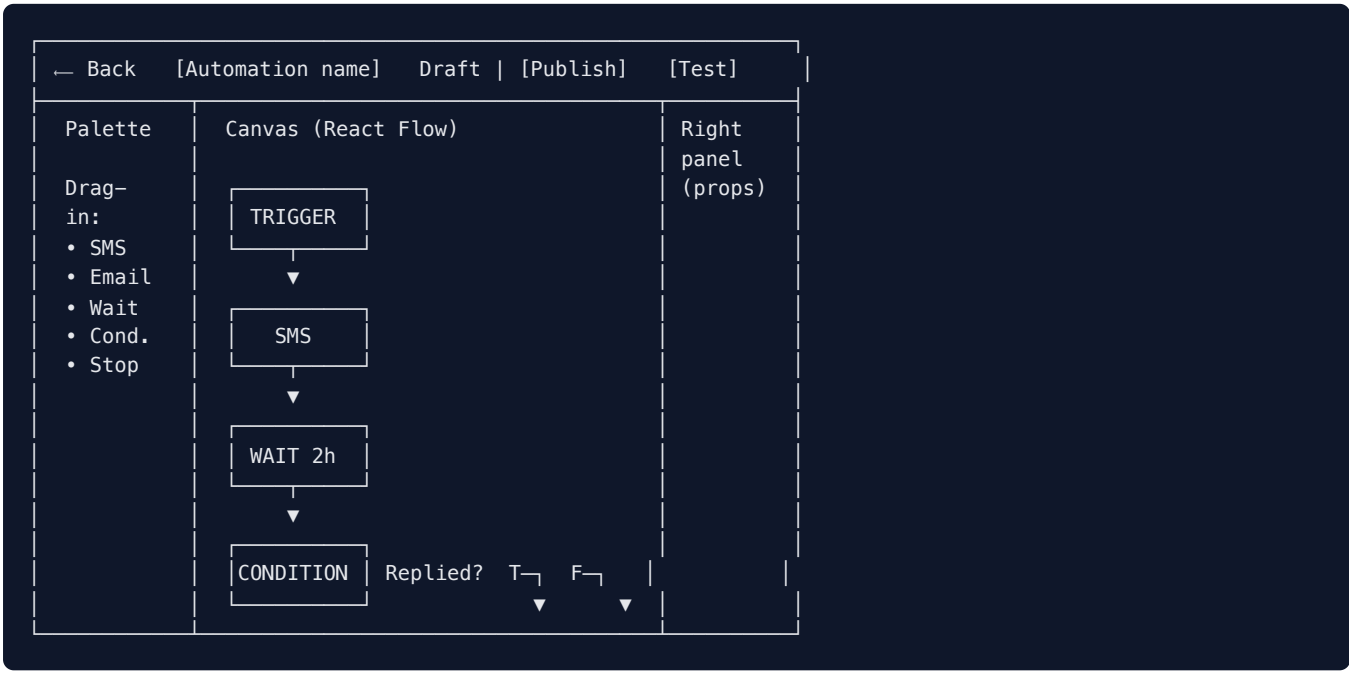
- `whatsapp` — same shape as `sms`
- `move_stage` — `{ to_stage }`
- `assign_staff` — `{ user_id | rule: 'round_robin' | 'least_loaded' }`
- `create_task`
- `webhook` — `{ url, method, body, headers }`
- `generate_doc` — `{ template_type: 'agreement' }`

7.3 Drag-Drop Flow Builder

Route: `apps/portal/src/app/(dashboard)/automations/[id]/page.tsx`

Library: **React Flow** (`@xyflow/react`). Already in NPM registry; install via `npm i @xyflow/react`.

UI



Node types (React Flow custom nodes)

- `TriggerNode` — root, single, fixed at top. Configures trigger type + filters.
- `SmsNode`, `EmailNode`, `WaitNode`, `ConditionNode`, `StopNode`.

Right panel (properties)

Shows form for the selected node. RHF + Zod per node type. Inline preview where applicable (e.g. SMS template rendered with sample data).

Auto-save

Drafts auto-save on every edit (debounced 500ms). No "Save" button — only "Publish".

Test mode

Button at top: **Test**.

- Choose a real lead (typeahead).
- The runtime fetches the lead's actual payload, executes the automation **without sending real SMS/email** — instead, returns step-by-step previews of what would be sent.
- Wait steps are simulated (return *"would wait 2h"*).
- Conditions evaluate against the real entity.
- Result: a timeline of what would happen.

7.4 Draft / Publish / Versioning

States

- `draft` — editable, not listening for events.
- `published` — listening for events. Editing puts the automation BACK into draft (`status='draft'`) but the **previously published version** continues to run for in-flight runs.
- `archived` — not listening; existing runs finish.

Versioning

- Every Publish snapshots the current draft into `automations.published_snapshot` (JSONB) and increments `version`.
- `automation_runs.automation_version` records which version the run is on.
- During execution, the engine reads steps from `published_snapshot` (not the live `automation_steps` table) so editing a published automation never breaks an in-flight run.

Publish flow

Edge function: `automation-publish`

1. Validate the automation has at least one step.
2. Build the snapshot: `{ trigger_type, trigger_config, steps: [...] }`.
3. Validate snapshot: no orphan steps, conditions have both branches, no cycles, no missing references.
4. Update `automations`:
   - `status='published'`
   - `published_at=NOW()`
   - `published_snapshot=<snapshot>`
   - `version=version+1` (only if was previously published; otherwise version=1)
5. Insert `lead_activity`-like audit row (re-use a new `automation_audit` table or generic audit).

Archive flow

- Set `status='archived'`. No new runs created. In-flight runs continue to completion.

7.5 Execution Engine

Event emission

Any place in the code that wants to fire an event MUST call a shared helper:

```
// supabase/functions/_shared/emit-event.ts
export async function emitEvent(
  supabase: SupabaseClient,
  params: {
    tenantId: string;
    eventType: string;   // e.g. 'lead.created'
    entityType: string;  // 'lead'
    entityId: string;
    payload: Record<string, unknown>;
  }
): Promise<void> {
  await supabase.from('automation_event_queue').insert({
    tenant_id: params.tenantId,
    event_type: params.eventType,
    entity_type: params.entityType,
    entity_id: params.entityId,
    payload: params.payload,
    processed: false,
  });
}
```

DB triggers on `leads.stage` UPDATE also call this via PL/pgSQL (call to a `notify_automation_event(...)` function).

Event processor

Cron edge function: `automation-poll-pending` — runs every minute.

Steps:

1. Pull unprocessed events: `SELECT * FROM automation_event_queue WHERE processed=false ORDER BY created_at LIMIT 100`.
2. For each event:
   1. Find published automations matching `tenant_id` + `trigger_type`.
   2. For each match, evaluate `trigger_config` filters against payload.
   3. For each passing automation, create `automation_runs` row with `status='running'`, `current_step_id=<first step>`.
   4. Execute first step via `automation-execute-step`.
3. Mark event row `processed=true`.

Step executor

Edge function: `automation-execute-step`

Inputs: `{ runId }`

Logic:

1. Load run + load step from `published_snapshot`.
2. Render variables in step config (using entity data + payload).
3. Execute according to `step_type`:
   - `sms` → call existing `aws-sns-sms` or Twilio (via existing helper). Record `automation_run_logs`. Move to next step.
   - `email` → call `aws-ses-email` or `resend-service`. Record. Next.
   - `wait` → set `automation_runs.status='waiting'`, `resume_at=NOW()+duration`. Stop.
   - `condition` → evaluate. Branch to true/false child. If no child, complete run.
   - `stop` → set `status='completed'`.
4. If next step exists, re-invoke `automation-execute-step` for the next step (or just continue inline).
5. If no next step, set `status='completed'`.

Wait resumption

Same `automation-poll-pending` cron job also scans:

```
SELECT * FROM automation_runs
WHERE status='waiting' AND resume_at <= NOW()
LIMIT 100;
```

For each, mark `status='running'`, advance to next step, execute.

Run cancellation

- Lead conversion → all running automations on that lead are marked `status='stopped'`.
- Lead deletion → cascade (`automation_runs.entity_id` → `ON DELETE CASCADE` semantically; implement via app-level cleanup since FK is generic).
- Operator pauses a run → `status='paused'`. Cron skips paused runs.

7.6 Attach to Lead

Event-driven (default)

Most automations have a trigger that fires automatically. The dropdown in the lead workspace shows the **manual-trigger** automations OR allows force-starting an event-driven one.

Manual attach UI

In right column, Automations section:

```
+ Attach automation ▾
   • Welcome sequence (manual)
   • Approved welcome pack (manual)
   • 5-day nurture (manual)
   • Lost-lead winback (manual)
   ── existing event-driven ──
   • Auto: stale 48h
   • Auto: stage changed → approved
```

On select → confirm modal → POST `automation-trigger-event` with `event_type='manual'`, `entity_type='lead'`, `entity_id=<lead>`, `automation_id=<specific>`.

Engine creates the run immediately and starts step 1.

---

8. Data Model

8.1 New Tables

All migrations live in `supabase/migrations/`. Naming: `YYYYMMDDHHMMSS_description.sql`.

Suggested order (split into multiple migrations for atomicity):

1. `20260601100000_create_blacklist_entries.sql`
2. `20260601101000_create_leads.sql`
3. `20260601102000_create_lead_notes_documents_activity.sql`
4. `20260601103000_create_conversations.sql`
5. `20260601104000_create_lead_offers.sql`
6. `20260601105000_create_automations.sql`
7. `20260601106000_create_automation_event_queue.sql`
8. `20260601107000_tenant_feature_flags.sql`
9. `20260601108000_migrate_enquiries_to_leads.sql`

`blacklist_entries`

```
CREATE TABLE public.blacklist_entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,

  phone_normalised TEXT,
  email_lower TEXT,
  licence_number TEXT,
  full_name TEXT,

  reason TEXT NOT NULL,
  notes TEXT,

  added_by UUID REFERENCES public.app_users(id) ON DELETE SET NULL,
  source_lead_id UUID, -- soft FK (leads table created next)
  source_customer_id UUID REFERENCES public.customers(id) ON DELETE SET NULL,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT blacklist_entries_has_identifier_chk
    CHECK (phone_normalised IS NOT NULL OR email_lower IS NOT NULL OR licence_number IS NOT NULL)
);

CREATE INDEX idx_blacklist_tenant_phone ON public.blacklist_entries (tenant_id, phone_normalised) WHERE pho
CREATE INDEX idx_blacklist_tenant_email ON public.blacklist_entries (tenant_id, email_lower) WHERE email_lo
CREATE INDEX idx_blacklist_tenant_licence ON public.blacklist_entries (tenant_id, licence_number) WHERE lic

CREATE TRIGGER set_blacklist_entries_updated_at
  BEFORE UPDATE ON public.blacklist_entries
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.blacklist_entries ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Tenant staff view blacklist" ON public.blacklist_entries
  FOR SELECT USING (tenant_id = get_user_tenant_id() OR is_super_admin());
CREATE POLICY "Tenant staff insert blacklist" ON public.blacklist_entries
  FOR INSERT WITH CHECK (tenant_id = get_user_tenant_id() OR is_super_admin());
CREATE POLICY "Tenant staff update blacklist" ON public.blacklist_entries
  FOR UPDATE USING (tenant_id = get_user_tenant_id() OR is_super_admin());
CREATE POLICY "Tenant staff delete blacklist" ON public.blacklist_entries
  FOR DELETE USING (tenant_id = get_user_tenant_id() OR is_super_admin());
```

`leads`

```
CREATE TABLE public.leads (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  customer_id UUID REFERENCES public.customers(id) ON DELETE SET NULL,

  -- Identity
  full_name TEXT NOT NULL,
  email TEXT NOT NULL,
  phone TEXT NOT NULL,
  phone_normalised TEXT NOT NULL, -- generated by trigger
  email_lower TEXT NOT NULL,      -- generated by trigger

  -- Application payload (Section 6.2 schema)
  application_data JSONB NOT NULL DEFAULT '{}'::jsonb,

  -- Vehicle interest
  vehicle_id UUID REFERENCES public.vehicles(id) ON DELETE SET NULL,
  vehicle_class TEXT,
  start_date DATE,
  end_date DATE,
  rental_type TEXT, -- daily | weekly | monthly

  -- Pipeline
  stage TEXT NOT NULL DEFAULT 'new',
  stage_updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  -- Scoring
  lead_score INT,
  score_band TEXT, -- hot | warm | cold | risk
  score_reason JSONB,

  -- Source
  source TEXT NOT NULL,
  source_metadata JSONB,

  -- Assignment
  assigned_to UUID REFERENCES public.app_users(id) ON DELETE SET NULL,

  -- Activity tracking
  last_contacted_at TIMESTAMPTZ,
  last_message_at TIMESTAMPTZ,
  last_activity_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  -- Blacklist link
  blacklist_match_id UUID REFERENCES public.blacklist_entries(id) ON DELETE SET NULL,

  -- Tags
  tags TEXT[] NOT NULL DEFAULT '{}'::text[],

  -- Conversion
  converted_at TIMESTAMPTZ,
  converted_to_rental_id UUID REFERENCES public.rentals(id) ON DELETE SET NULL,

  -- Read tracking (mirror enquiries)
  is_read BOOLEAN NOT NULL DEFAULT false,
  read_at TIMESTAMPTZ,
  read_by UUID REFERENCES public.app_users(id) ON DELETE SET NULL,

  -- Audit
  ip_address INET,
  user_agent TEXT,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
  CONSTRAINT leads_stage_chk CHECK (stage IN (
    'new', 'contacted', 'docs_requested', 'docs_submitted', 'docs_verified', 'docs_failed',
    'approved', 'vehicle_offered', 'offer_accepted',
    'agreement_sent', 'agreement_signed', 'deposit_paid', 'pickup_scheduled',
    'converted', 'waitlist', 'lost', 'blacklisted'
  )),
  CONSTRAINT leads_score_band_chk
    CHECK (score_band IS NULL OR score_band IN ('hot', 'warm', 'cold', 'risk')),
  CONSTRAINT leads_source_chk CHECK (source IN (
    'application', 'quick_enquiry', 'phone_in', 'walk_in', 'ad_landing',
    'admin_manual', 'inbound_sms', 'inbound_email', 'inbound_whatsapp', 'legacy_enquiry'
  )),
  CONSTRAINT leads_dates_chk CHECK (end_date IS NULL OR start_date IS NULL OR end_date >= start_date)
);

CREATE INDEX idx_leads_tenant_stage_created ON public.leads (tenant_id, stage, created_at DESC);
CREATE INDEX idx_leads_tenant_assigned ON public.leads (tenant_id, assigned_to);
CREATE INDEX idx_leads_tenant_phone ON public.leads (tenant_id, phone_normalised);
CREATE INDEX idx_leads_tenant_email ON public.leads (tenant_id, email_lower);
CREATE INDEX idx_leads_tenant_score ON public.leads (tenant_id, score_band, lead_score DESC);
CREATE INDEX idx_leads_last_activity ON public.leads (tenant_id, last_activity_at DESC);
CREATE INDEX idx_leads_vehicle_id ON public.leads (vehicle_id) WHERE vehicle_id IS NOT NULL;

-- Normalisation trigger
CREATE OR REPLACE FUNCTION public.normalise_lead_identifiers()
RETURNS TRIGGER AS $$
BEGIN
  NEW.phone_normalised := regexp_replace(COALESCE(NEW.phone, ''), '\D', '', 'g');
  NEW.email_lower := lower(trim(COALESCE(NEW.email, '')));
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER normalise_lead_identifiers_trg
  BEFORE INSERT OR UPDATE OF phone, email ON public.leads
  FOR EACH ROW EXECUTE FUNCTION public.normalise_lead_identifiers();

CREATE TRIGGER set_leads_updated_at
  BEFORE UPDATE ON public.leads
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

-- Stage transition validation
CREATE OR REPLACE FUNCTION public.validate_lead_stage_transition()
RETURNS TRIGGER AS $$
BEGIN
  IF OLD.stage IS DISTINCT FROM NEW.stage THEN
    NEW.stage_updated_at := NOW();
    -- Allowed transitions enforced in app layer (validate_lead_stage_transition TS function).
    -- DB-level: just record stage_updated_at and let app validate.
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER validate_lead_stage_trg
  BEFORE UPDATE OF stage ON public.leads
  FOR EACH ROW EXECUTE FUNCTION public.validate_lead_stage_transition();

ALTER TABLE public.leads ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Tenant staff view leads" ON public.leads
  FOR SELECT USING (tenant_id = get_user_tenant_id() OR is_super_admin());
CREATE POLICY "Tenant staff update leads" ON public.leads
  FOR UPDATE USING (tenant_id = get_user_tenant_id() OR is_super_admin());
CREATE POLICY "Tenant staff delete leads" ON public.leads
```

```
  FOR DELETE USING (tenant_id = get_user_tenant_id() OR is_super_admin());
-- INSERT only via service_role (edge functions).

-- Realtime
ALTER PUBLICATION supabase_realtime ADD TABLE public.leads;
```

`lead_documents`

```
CREATE TABLE public.lead_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  lead_id UUID NOT NULL REFERENCES public.leads(id) ON DELETE CASCADE,

  document_type TEXT NOT NULL,
  file_url TEXT NOT NULL,
  file_name TEXT,
  file_size BIGINT,
  mime_type TEXT,

  verification_status TEXT NOT NULL DEFAULT 'pending',
  verification_id UUID,
  verification_error TEXT,
  expires_at DATE,

  uploaded_by_lead BOOLEAN NOT NULL DEFAULT true,
  uploaded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT lead_documents_type_chk CHECK (document_type IN (
    'licence', 'selfie', 'rideshare_proof', 'insurance', 'passport', 'utility_bill', 'other'
  )),
  CONSTRAINT lead_documents_verification_chk CHECK (verification_status IN (
    'pending', 'verifying', 'verified', 'failed', 'expired'
  ))
);

CREATE INDEX idx_lead_documents_lead ON public.lead_documents (lead_id);
CREATE INDEX idx_lead_documents_tenant ON public.lead_documents (tenant_id);

ALTER TABLE public.lead_documents ENABLE ROW LEVEL SECURITY;
-- Same RLS pattern (tenant_id = get_user_tenant_id() OR is_super_admin()) for SELECT/UPDATE/DELETE.
-- INSERT via service_role only.
```

`lead_notes`

```
CREATE TABLE public.lead_notes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  lead_id UUID NOT NULL REFERENCES public.leads(id) ON DELETE CASCADE,

  author_id UUID NOT NULL REFERENCES public.app_users(id) ON DELETE SET NULL,
  body TEXT NOT NULL,
  is_pinned BOOLEAN NOT NULL DEFAULT false,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_lead_notes_lead ON public.lead_notes (lead_id, created_at DESC);

CREATE TRIGGER set_lead_notes_updated_at
  BEFORE UPDATE ON public.lead_notes
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.lead_notes ENABLE ROW LEVEL SECURITY;
-- Standard tenant RLS.
```

`lead_activity`

```
CREATE TABLE public.lead_activity (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  lead_id UUID NOT NULL REFERENCES public.leads(id) ON DELETE CASCADE,

  actor_type TEXT NOT NULL, -- system | staff | lead | ai
  actor_id UUID,             -- app_user.id when staff; null otherwise

  event_type TEXT NOT NULL,
  payload JSONB NOT NULL DEFAULT '{}'::jsonb,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_lead_activity_lead_created ON public.lead_activity (lead_id, created_at DESC);

ALTER TABLE public.lead_activity ENABLE ROW LEVEL SECURITY;
-- Standard tenant RLS (SELECT only for staff; INSERT via service_role).
```

`conversations`

```
CREATE TABLE public.conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,

  lead_id UUID REFERENCES public.leads(id) ON DELETE CASCADE,
  customer_id UUID REFERENCES public.customers(id) ON DELETE CASCADE,

  last_message_at TIMESTAMPTZ,
  unread_count INT NOT NULL DEFAULT 0,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT conversation_subject_chk
    CHECK (lead_id IS NOT NULL OR customer_id IS NOT NULL)
);

CREATE UNIQUE INDEX idx_conversations_lead ON public.conversations (lead_id) WHERE lead_id IS NOT NULL;
CREATE INDEX idx_conversations_customer ON public.conversations (customer_id) WHERE customer_id IS NOT NULL
CREATE INDEX idx_conversations_tenant_last ON public.conversations (tenant_id, last_message_at DESC NULLS LA

CREATE TRIGGER set_conversations_updated_at
  BEFORE UPDATE ON public.conversations
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.conversations ENABLE ROW LEVEL SECURITY;
-- Standard tenant RLS.

ALTER PUBLICATION supabase_realtime ADD TABLE public.conversations;
```

`conversation_messages`

```
CREATE TABLE public.conversation_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  conversation_id UUID NOT NULL REFERENCES public.conversations(id) ON DELETE CASCADE,

  channel TEXT NOT NULL,           -- sms | email | whatsapp | in_app | note | system | call_summary
  direction TEXT NOT NULL,         -- inbound | outbound | internal
  sender_type TEXT NOT NULL,       -- lead | customer | staff | system | ai
  sender_id UUID,

  body TEXT,
  subject TEXT,
  attachments JSONB NOT NULL DEFAULT '[]'::jsonb,

  channel_message_id TEXT,         -- Twilio SID, email Message-ID, etc.
  status TEXT NOT NULL DEFAULT 'sent',  -- queued | sent | delivered | read | failed
  error TEXT,

  read_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT conversation_messages_channel_chk CHECK (channel IN (
    'sms', 'email', 'whatsapp', 'in_app', 'note', 'system', 'call_summary'
  )),
  CONSTRAINT conversation_messages_direction_chk CHECK (direction IN (
    'inbound', 'outbound', 'internal'
  )),
  CONSTRAINT conversation_messages_sender_type_chk CHECK (sender_type IN (
    'lead', 'customer', 'staff', 'system', 'ai'
  ))
);

CREATE INDEX idx_conv_messages_conv_created ON public.conversation_messages (conversation_id, created_at DE
CREATE INDEX idx_conv_messages_tenant_created ON public.conversation_messages (tenant_id, created_at DESC);
CREATE INDEX idx_conv_messages_channel_id ON public.conversation_messages (channel_message_id) WHERE channe

ALTER TABLE public.conversation_messages ENABLE ROW LEVEL SECURITY;
-- Standard tenant RLS.

ALTER PUBLICATION supabase_realtime ADD TABLE public.conversation_messages;
```

`lead_offers`

```
CREATE TABLE public.lead_offers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  lead_id UUID NOT NULL REFERENCES public.leads(id) ON DELETE CASCADE,

  short_code TEXT NOT NULL UNIQUE,    -- 8-12 char URL-safe
  vehicles JSONB NOT NULL,             -- [{ vehicle_id, price_override?, start_date, end_date, kind: 'sing
  custom_message TEXT,
  default_start_date DATE NOT NULL,
  default_end_date DATE NOT NULL,
  date_flex_days INT NOT NULL DEFAULT 0,
  deposit_amount INT,
  show_prices BOOLEAN NOT NULL DEFAULT true,

  expires_at TIMESTAMPTZ NOT NULL,

  status TEXT NOT NULL DEFAULT 'pending',  -- pending | viewed | accepted | declined | expired
  view_count INT NOT NULL DEFAULT 0,
  first_viewed_at TIMESTAMPTZ,
  last_viewed_at TIMESTAMPTZ,

  accepted_vehicle_id UUID REFERENCES public.vehicles(id) ON DELETE SET NULL,
  accepted_start_date DATE,
  accepted_end_date DATE,
  accepted_at TIMESTAMPTZ,

  created_by UUID REFERENCES public.app_users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT lead_offers_status_chk CHECK (status IN ('pending', 'viewed', 'accepted', 'declined', 'expired
  CONSTRAINT lead_offers_dates_chk CHECK (default_end_date >= default_start_date)
);

CREATE INDEX idx_lead_offers_lead ON public.lead_offers (lead_id, created_at DESC);
CREATE INDEX idx_lead_offers_tenant_status ON public.lead_offers (tenant_id, status);
-- short_code already unique-indexed via UNIQUE constraint.

CREATE TRIGGER set_lead_offers_updated_at
  BEFORE UPDATE ON public.lead_offers
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.lead_offers ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Tenant staff view offers" ON public.lead_offers
  FOR SELECT USING (tenant_id = get_user_tenant_id() OR is_super_admin());
CREATE POLICY "Tenant staff update offers" ON public.lead_offers
  FOR UPDATE USING (tenant_id = get_user_tenant_id() OR is_super_admin());
-- INSERT and lead-side SELECT-by-code via service_role only.

ALTER PUBLICATION supabase_realtime ADD TABLE public.lead_offers;
```

`automations`

```
CREATE TABLE public.automations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,

  name TEXT NOT NULL,
  description TEXT,

  trigger_type TEXT NOT NULL,
  trigger_config JSONB NOT NULL DEFAULT '{}'::jsonb,

  status TEXT NOT NULL DEFAULT 'draft',  -- draft | published | archived
  version INT NOT NULL DEFAULT 0,
  published_at TIMESTAMPTZ,
  published_snapshot JSONB,   -- frozen { trigger_type, trigger_config, steps[] }

  created_by UUID REFERENCES public.app_users(id) ON DELETE SET NULL,
  updated_by UUID REFERENCES public.app_users(id) ON DELETE SET NULL,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT automations_status_chk CHECK (status IN ('draft', 'published', 'archived'))
);

CREATE INDEX idx_automations_tenant_status ON public.automations (tenant_id, status);
CREATE INDEX idx_automations_trigger ON public.automations (tenant_id, trigger_type, status) WHERE status =

CREATE TRIGGER set_automations_updated_at
  BEFORE UPDATE ON public.automations
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.automations ENABLE ROW LEVEL SECURITY;
-- Tenant RLS.
```

`automation_steps`

```
CREATE TABLE public.automation_steps (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  automation_id UUID NOT NULL REFERENCES public.automations(id) ON DELETE CASCADE,
  parent_step_id UUID REFERENCES public.automation_steps(id) ON DELETE CASCADE,
  order_index INT NOT NULL,
  step_type TEXT NOT NULL,    -- sms | email | wait | condition | stop
  config JSONB NOT NULL,
  branch TEXT,                 -- 'true' | 'false' (only when parent is condition)
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT automation_steps_type_chk CHECK (step_type IN ('sms', 'email', 'wait', 'condition', 'stop')),
  CONSTRAINT automation_steps_branch_chk CHECK (branch IS NULL OR branch IN ('true', 'false'))
);

CREATE INDEX idx_automation_steps_automation ON public.automation_steps (automation_id, order_index);

CREATE TRIGGER set_automation_steps_updated_at
  BEFORE UPDATE ON public.automation_steps
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.automation_steps ENABLE ROW LEVEL SECURITY;
-- Inherit tenant via automation_id (subquery).
CREATE POLICY "Tenant view automation steps" ON public.automation_steps
  FOR SELECT USING (
    automation_id IN (SELECT id FROM public.automations WHERE tenant_id = get_user_tenant_id())
    OR is_super_admin()
  );
-- Similar for UPDATE / DELETE / INSERT (via service_role only).
```

automation_runs

```
CREATE TABLE public.automation_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  automation_id UUID NOT NULL REFERENCES public.automations(id),
  automation_version INT NOT NULL,

  entity_type TEXT NOT NULL,   -- lead | customer | rental | …
  entity_id UUID NOT NULL,

  status TEXT NOT NULL DEFAULT 'running',  -- running | waiting | paused | completed | stopped | failed
  current_step_id UUID,
  resume_at TIMESTAMPTZ,

  triggered_by TEXT NOT NULL,    -- event | manual
  triggered_by_event TEXT,
  triggered_by_user UUID REFERENCES public.app_users(id) ON DELETE SET NULL,

  context JSONB NOT NULL DEFAULT '{}'::jsonb,
  error_message TEXT,

  started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ended_at TIMESTAMPTZ,

  CONSTRAINT automation_runs_status_chk CHECK (status IN (
    'running', 'waiting', 'paused', 'completed', 'stopped', 'failed'
  ))
);

CREATE INDEX idx_automation_runs_entity ON public.automation_runs (entity_type, entity_id);
CREATE INDEX idx_automation_runs_pending ON public.automation_runs (status, resume_at) WHERE status IN ('ru
CREATE INDEX idx_automation_runs_tenant ON public.automation_runs (tenant_id, status);

ALTER TABLE public.automation_runs ENABLE ROW LEVEL SECURITY;
-- Tenant RLS.
ALTER PUBLICATION supabase_realtime ADD TABLE public.automation_runs;
```

automation_run_logs

```
CREATE TABLE public.automation_run_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID NOT NULL REFERENCES public.automation_runs(id) ON DELETE CASCADE,
  step_id UUID REFERENCES public.automation_steps(id) ON DELETE SET NULL,

  status TEXT NOT NULL,         -- executed | skipped | failed
  output JSONB,
  error TEXT,

  executed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT automation_run_logs_status_chk CHECK (status IN ('executed', 'skipped', 'failed'))
);

CREATE INDEX idx_automation_run_logs_run ON public.automation_run_logs (run_id, executed_at);

ALTER TABLE public.automation_run_logs ENABLE ROW LEVEL SECURITY;
-- Inherit via run_id → tenant.
```

automation_event_queue

```
CREATE TABLE public.automation_event_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,

  event_type TEXT NOT NULL,
  entity_type TEXT NOT NULL,
  entity_id UUID NOT NULL,
  payload JSONB NOT NULL DEFAULT '{}'::jsonb,

  processed BOOLEAN NOT NULL DEFAULT false,
  processed_at TIMESTAMPTZ,
  attempts INT NOT NULL DEFAULT 0,
  last_error TEXT,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_automation_event_queue_pending ON public.automation_event_queue (processed, created_at) WH
CREATE INDEX idx_automation_event_queue_entity ON public.automation_event_queue (entity_type, entity_id, cr

ALTER TABLE public.automation_event_queue ENABLE ROW LEVEL SECURITY;
-- Tenant SELECT; INSERT via service_role only.
```

lead_message_templates

```
CREATE TABLE public.lead_message_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,

  name TEXT NOT NULL,
  channel TEXT NOT NULL,        -- sms | email | whatsapp
  category TEXT NOT NULL,       -- welcome | doc_request | approval | offer | reminder | decline | followup
  subject TEXT,                 -- for email
  body TEXT NOT NULL,           -- supports {{variables}}
  is_default BOOLEAN NOT NULL DEFAULT false,
  is_active BOOLEAN NOT NULL DEFAULT true,

  created_by UUID REFERENCES public.app_users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT lead_msg_templates_channel_chk CHECK (channel IN ('sms', 'email', 'whatsapp'))
);

CREATE INDEX idx_lead_msg_templates_tenant ON public.lead_message_templates (tenant_id, channel, category);

CREATE TRIGGER set_lead_msg_templates_updated_at
  BEFORE UPDATE ON public.lead_message_templates
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

ALTER TABLE public.lead_message_templates ENABLE ROW LEVEL SECURITY;
-- Tenant RLS.
```

8.2 Modified Tables

`tenants`

```
ALTER TABLE public.tenants
  ADD COLUMN lead_management_enabled BOOLEAN NOT NULL DEFAULT false,
  ADD COLUMN automations_enabled BOOLEAN NOT NULL DEFAULT false,
  ADD COLUMN lead_stale_threshold_hours INT NOT NULL DEFAULT 48,
  ADD COLUMN lead_auto_lost_threshold_hours INT NOT NULL DEFAULT 168;  -- 7 days
```

`enquiries` migration

Migration `20260601108000_migrate_enquiries_to_leads.sql`:

```
-- Copy existing enquiries into leads as source='legacy_enquiry'
INSERT INTO public.leads (
  id, tenant_id, customer_id,
  full_name, email, phone,
  vehicle_id, start_date, end_date,
  stage, source, source_metadata,
  is_read, read_at, read_by,
  ip_address, user_agent,
  application_data,
  created_at, updated_at
)
SELECT
  id, tenant_id, customer_id,
  customer_name, customer_email, customer_phone,
  vehicle_id, start_date, end_date,
  CASE
    WHEN status = 'new' THEN 'new'
    WHEN status = 'contacted' THEN 'contacted'
    WHEN status = 'resolved' THEN 'lost'   -- legacy "resolved" = closed; map to lost
    ELSE 'new'
  END,
  'legacy_enquiry', jsonb_build_object('legacy_status', status, 'description', description),
  is_read, read_at, read_by,
  ip_address, user_agent,
  jsonb_build_object('description', description),
  created_at, updated_at
FROM public.enquiries
ON CONFLICT (id) DO NOTHING;

-- Keep enquiries table in place for backward compatibility but mark deprecated.
COMMENT ON TABLE public.enquiries IS 'DEPRECATED — migrated into public.leads on 2026-06-01. Will be droppe
```

> The new "Quick Enquiry" submissions still write to `leads` (`source='quick_enquiry'`) — the existing `submit-enquiry` edge function is refactored to write to `leads` instead of `enquiries`.

8.3 RLS Strategy Summary

Use the existing helper functions everywhere:

- `get_user_tenant_id()` — tenant scope.
- `is_super_admin()` — bypass.
- `is_primary_super_admin()` / `is_global_master_admin()` — not needed for this feature.

Standard policy template for tenant-scoped tables:

```
CREATE POLICY "Tenant staff view X" ON public.X
  FOR SELECT USING (tenant_id = get_user_tenant_id() OR is_super_admin());

CREATE POLICY "Tenant staff update X" ON public.X
  FOR UPDATE USING (tenant_id = get_user_tenant_id() OR is_super_admin());

CREATE POLICY "Tenant staff delete X" ON public.X
  FOR DELETE USING (tenant_id = get_user_tenant_id() OR is_super_admin());

-- INSERT only via service_role from edge functions — no public policy.
```

For nested tables (`automation_steps` , `automation_run_logs`), inherit via subquery on the parent.

8.4 Database Functions & Triggers

`set_updated_at()`

Already exists. Reuse for every `updated_at` column.

`normalise_lead_identifiers()`

New. See `leads` table SQL above.

`validate_lead_stage_transition()`

New. See `leads` table SQL above.

`notify_automation_event()`

New trigger function — called from row-level triggers on key tables.

```
CREATE OR REPLACE FUNCTION public.notify_automation_event(
  p_event_type TEXT,
  p_tenant_id UUID,
  p_entity_type TEXT,
  p_entity_id UUID,
  p_payload JSONB DEFAULT '{}'::jsonb
)
RETURNS VOID AS $$
BEGIN
  INSERT INTO public.automation_event_queue (
    tenant_id, event_type, entity_type, entity_id, payload
  ) VALUES (
    p_tenant_id, p_event_type, p_entity_type, p_entity_id, p_payload
  );
END;
$$ LANGUAGE plpgsql;
```

Call sites:

```
-- On leads INSERT:
CREATE OR REPLACE FUNCTION public.emit_lead_created() RETURNS TRIGGER AS $$
BEGIN
  PERFORM public.notify_automation_event(
    'lead.created', NEW.tenant_id, 'lead', NEW.id,
    jsonb_build_object('source', NEW.source, 'score_band', NEW.score_band)
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER emit_lead_created_trg AFTER INSERT ON public.leads
  FOR EACH ROW EXECUTE FUNCTION public.emit_lead_created();

-- On leads UPDATE of stage:
CREATE OR REPLACE FUNCTION public.emit_lead_stage_changed() RETURNS TRIGGER AS $$
BEGIN
  IF OLD.stage IS DISTINCT FROM NEW.stage THEN
    PERFORM public.notify_automation_event(
      'lead.stage_changed', NEW.tenant_id, 'lead', NEW.id,
      jsonb_build_object('from_stage', OLD.stage, 'to_stage', NEW.stage)
    );
    -- Also emit terminal-specific events:
    IF NEW.stage = 'lost' THEN
      PERFORM public.notify_automation_event('lead.lost', NEW.tenant_id, 'lead', NEW.id, '{}'::jsonb);
    ELSIF NEW.stage = 'blacklisted' THEN
      PERFORM public.notify_automation_event('lead.blacklisted', NEW.tenant_id, 'lead', NEW.id, '{}'::jsonb
    ELSIF NEW.stage = 'converted' THEN
      PERFORM public.notify_automation_event(
        'lead.converted', NEW.tenant_id, 'lead', NEW.id,
        jsonb_build_object('customer_id', NEW.customer_id, 'rental_id', NEW.converted_to_rental_id)
      );
    END IF;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER emit_lead_stage_changed_trg AFTER UPDATE OF stage ON public.leads
  FOR EACH ROW EXECUTE FUNCTION public.emit_lead_stage_changed();
```

Similar triggers for `lead_documents`, `lead_offers`, etc.

8.5 Realtime Publications

Add these tables to `supabase_realtime` publication:

- `leads`
- `conversations`
- `conversation_messages`
- `lead_offers`
- `automation_runs`

Done via `ALTER PUBLICATION supabase_realtime ADD TABLE public.<name>;` in each migration.

---

9. Edge Functions

9.1 New Functions

All under `supabase/functions/<name>/index.ts`. Use `_shared/cors.ts` helpers as existing.

Function	JWT	Purpose
`submit-application`	No	Public submission from `/apply`. Validates, dedups, blacklist-checks, scores, creates lead.
`submit-quick-enquiry`	No	Replaces existing `submit-enquiry`. Same shape but writes to `leads` (`source='quick_enquiry'`).
`check-blacklist-match`	Yes	Returns blacklist match result for given identifiers.
`compute-lead-score`	Yes	Computes score + band from a lead's application_data + history.
`run-matching-engine`	Yes	Returns ranked vehicle matches for a lead.
`create-offer-link`	Yes	Builds an offer + dispatches via chosen channel.
`view-offer`	No	Public: resolves offer by short_code, returns payload. Tracks views.
`accept-offer`	No	Public: lead picks a vehicle, transitions lead.
`convert-lead-to-rental`	Yes	Lead → customer → rental conversion.
`send-lead-message`	Yes	Multi-channel send (SMS/email/WhatsApp) into a lead conversation.
`inbound-sms-webhook`	No (Twilio signature)	Receives Twilio inbound SMS + WhatsApp.
`inbound-email-webhook`	No (provider signature)	Receives SES/Resend inbound.
`lead-document-presign`	Yes (or magic-link token)	Returns presigned upload URL for lead documents.
`ai-extract-from-conversation`	Yes	LLM extracts structured fields from conversation thread.
`ai-rank-matches`	Yes	LLM reranks matching engine results.
`ai-suggest-next-action`	Yes	LLM proposes next action for a lead.
`ai-draft-message`	Yes	LLM drafts a message in tenant tone.
`automation-trigger-event`	Yes / internal	Queues an event (callable from app code OR DB trigger fallback).
`automation-execute-step`	Internal (service_role)	Runs one step of a run.
`automation-poll-pending`	Cron (no JWT, IP-restricted)	Picks up queue events + resumes waiting runs.
`automation-publish`	Yes	Snapshots draft → published.

Add corresponding entries in `supabase/config.toml` for `verify_jwt = false` where needed. Mirror existing webhook pattern.

9.2 Reused Functions

Existing function	Use in this feature
`create-veriff-session`	Lead doc verification — fired from Quick Action button or auto-fire on `docs_submitted`
`create-ai-verification-session`	Same
`ai-document-ocr`	Same
`ai-face-match`	Same
`bonzah-create-quote`, `bonzah-confirm-payment`	Insurance qualification step
`aws-ses-email`	Email send within `send-lead-message`
`aws-sns-sms`	SMS send (primary) within `send-lead-message`
`send-collection-whatsapp`	WhatsApp send (reuse same Twilio infra) within `send-lead-message`
`send-signing-email`, `send-signing-whatsapp`	Agreement send — triggered from Quick Action
`create-checkout-session`, `create-preauth-checkout`	Deposit payment links from Quick Action
BoldSign helpers (`_shared/boldsign-client.ts`)	Agreement generation
`_shared/cors.ts`	All new edge functions
Existing template render service	Reuse for `{{variables}}` substitution

9.3 Function Specifications (key ones)

`submit-application`

```
POST /functions/v1/submit-application
Headers:
  X-Tenant-Slug: <tenant_slug>     (or rely on host)
  Content-Type: application/json
Body: ApplyFormValues (Section 6.2)

Response 200:
  { leadId: string; status: 'received' | 'duplicate_merged' | 'blacklisted' }

Side effects:
  - Insert lead row (or merge into existing if dedup hits)
  - Insert lead_documents rows
  - Insert conversation row
  - Acknowledge SMS sent (hardcoded V1 template)
  - automation_event_queue entries for lead.created + lead.application_submitted
```

Errors:

- 400 — validation
- 403 — `lead_management_enabled = false` for tenant
- 429 — rate limit

`run-matching-engine`

```
POST /functions/v1/run-matching-engine
Headers: Authorization (JWT — staff)
Body: { leadId: string } | MatchInput

Response 200: MatchResult (Section 6.5)

Implementation notes:
  - Use shared TS helper supabase/functions/_shared/matching.ts
  - Cache results in lead.matching_cache JSONB column? — NO in V1 (keep simple, recompute on demand)
  - Calls ai-rank-matches inline if AI features enabled for tenant
```

`create-offer-link`

```
POST /functions/v1/create-offer-link
Headers: Authorization (JWT — staff with leads.editor)
Body: {
  leadId: string;
  vehicles: Array<{ vehicleId: string; priceOverride?: number; startDate: string; endDate: string }>;
  customMessage?: string;
  defaultStartDate: string;
  defaultEndDate: string;
  dateFlexDays: number;
  depositAmount?: number;
  showPrices: boolean;
  expiresInHours: number;
  sendMethod: 'sms' | 'email' | 'whatsapp' | 'copy';
}

Response 200: { offerId: string; shortCode: string; url: string }

Side effects:
  - lead_offers row inserted
  - lead.stage → 'vehicle_offered'
  - If sendMethod ≠ 'copy': send-lead-message dispatch
  - automation_event_queue: lead.offer_sent
  - lead_activity: 'offer_sent'
```

`accept-offer`

```
POST /functions/v1/accept-offer
Headers: (none — public)
Body: {
  shortCode: string;
  vehicleId: string;
  startDate: string;
  endDate: string;
}

Response 200: { status: 'accepted'; offerId: string }
Response 409: { status: 'vehicle_unavailable'; available_vehicles: [...] }
Response 410: { status: 'expired' }
Response 404: { status: 'not_found' }

Side effects:
  - lead_offers updated (accepted_*)
  - lead.stage → 'offer_accepted'
  - automation_event_queue: lead.offer_accepted
  - lead_activity: 'offer_accepted'
```

`automation-poll-pending`

```
GET /functions/v1/automation-poll-pending
Headers: X-Cron-Secret: <env.AUTOMATION_CRON_SECRET>
(Triggered by Supabase Scheduler every 60s)

Behaviour:
  1. Process up to 100 unprocessed events:
     - Find published automations matching event_type + trigger_config filter
     - For each match, INSERT automation_runs with status='running', current_step_id=<first>
     - Mark event processed
  2. Resume up to 100 waiting runs (resume_at <= NOW):
     - Set status='running'
     - Execute current step via automation-execute-step

Response: { processed: N, resumed: M, errors: [...] }
```

Schedule: `0 * * * * *` (every minute). Use Supabase `pg_cron` or external scheduler.

`automation-execute-step`

```
Internal only — invoked from automation-poll-pending or after a prior step.

Inputs: { runId: string }

Logic:
  1. Load automation_runs row, validate status='running'
  2. Load step config from automations.published_snapshot
  3. Render template variables
  4. Execute by step_type
  5. Insert automation_run_logs
  6. If next step exists, set current_step_id, optionally re-invoke (or return for caller to loop)
  7. If no next step, set status='completed', ended_at=NOW
```

---

10. Frontend Structure

10.1 Portal Routes

```
apps/portal/src/app/(dashboard)/
  leads/
    page.tsx                          # Kanban board (Section 6.3)
    [id]/
      page.tsx                        # 3-column workspace (Section 6.4)
  automations/
    page.tsx                          # List (Section 7)
    [id]/
      page.tsx                        # Flow builder (Section 7.3)
  settings/
    lead-management/page.tsx          # Tenant settings (Section 14)
    automations/page.tsx              # Optional: automation settings
```

10.2 Portal Components

```
apps/portal/src/components/leads/
  lead-board.tsx                      # Kanban container
  lead-board-column.tsx               # One column
  lead-card.tsx                       # Card in column
  lead-card-preview.tsx               # Hover preview
  lead-workspace.tsx                  # 3-column shell
  lead-info-panel.tsx                 # Left
  lead-communication-panel.tsx        # Middle
  lead-message-bubble.tsx
  lead-composer.tsx                   # Bottom of middle
  lead-template-picker.tsx
  lead-ai-panel.tsx                   # Right (parent)
  lead-ai-next-action.tsx             # Right - section 1
  lead-matching-engine.tsx            # Right - section 2
  lead-automations-panel.tsx          # Right - section 3
  offer-builder-dialog.tsx
  blacklist-confirm-dialog.tsx
  convert-to-rental-dialog.tsx
  request-documents-dialog.tsx
  lead-notes-list.tsx
  lead-activity-timeline.tsx
  lead-documents-list.tsx
  lead-tag-input.tsx

apps/portal/src/components/automations/
  automation-list.tsx                 # /automations index
  automation-list-card.tsx
  flow-builder.tsx                    # React Flow canvas
  flow-builder-palette.tsx
  flow-builder-properties.tsx
  flow-node-trigger.tsx
  flow-node-sms.tsx
  flow-node-email.tsx
  flow-node-wait.tsx
  flow-node-condition.tsx
  flow-node-stop.tsx
  trigger-picker.tsx
  publish-dialog.tsx
  test-run-dialog.tsx
  test-run-timeline.tsx
```

10.3 Portal Hooks

```
apps/portal/src/hooks/
  use-leads.ts                        # Board list, with filters
  use-lead.ts                         # Single lead by id
  use-lead-mutations.ts               # stage change, assign, tags, notes
  use-lead-board.ts                   # Wrap useLeads for board (grouped by stage)
  use-conversation.ts                 # For a lead/customer
  use-conversation-messages.ts        # With realtime
  use-send-message.ts                 # Mutation
  use-matching-engine.ts
  use-offer-link.ts                   # Create/list offers for a lead
  use-blacklist.ts                    # List/add/remove
  use-lead-templates.ts               # Templates
  use-lead-documents.ts
  use-lead-notes.ts
  use-lead-activity.ts

  use-automations.ts                  # List
  use-automation.ts                   # Single + draft edit
  use-automation-runs.ts              # Runs for an entity
  use-automation-publish.ts           # Publish mutation
  use-automation-trigger-registry.ts  # Static event list

  use-ai-extract.ts                   # Conversation → structured
  use-ai-suggest.ts                   # Next action
  use-ai-draft.ts                     # Message draft
```

React Query patterns:

- Query keys include `tenant?.id`
- `staleTime: 60_000` default
- Realtime subscriptions invalidate query data via custom hook `useRealtimeInvalidate(channel, queryKey)`

10.4 Portal Lib

```
apps/portal/src/lib/
  lead-stage-machine.ts               # canTransition, allowedTransitions, stageLabel, stageColor
  automation-event-registry.ts        # Event list with payload schemas
  automation-step-validators.ts       # Zod schemas per step type
  template-variables.ts               # Variable catalog for autocomplete
  matching-types.ts                   # Shared types (mirror of edge fn)
  permissions.ts                      # Extend with new tab keys
```

10.5 Portal Stores

No new stores needed — React Query + URL state cover everything. Existing `auth-store.ts`, `settings-store.ts` unchanged.

10.6 Booking Routes

```
apps/booking/src/app/
  apply/
    page.tsx                          # Multi-step form (Section 6.2)
    submitted/page.tsx                # Confirmation
  offer/
    [code]/page.tsx                   # Customer offer page (Section 6.6)
    [code]/accepted/page.tsx          # Acceptance confirmation
```

10.7 Booking Components

```
apps/booking/src/components/apply/
  apply-form.tsx                    # Wizard shell
  apply-progress.tsx                # Step bar
  step-1-about.tsx
  step-2-driver.tsx
  step-3-intent.tsx
  step-4-financial.tsx
  step-5-history.tsx
  step-6-documents.tsx
  step-7-review.tsx

apps/booking/src/components/offer/
  offer-page.tsx                    # Layout
  offer-vehicle-card.tsx
  offer-date-picker.tsx             # Constrained to ± flex days
  offer-accepted-screen.tsx
  offer-expired-screen.tsx
```

10.8 Booking Hooks / Schemas

```
apps/booking/src/client-schemas/
  apply.ts                          # Section 6.2 schema

apps/booking/src/hooks/
  use-apply-submit.ts
  use-offer.ts                      # Fetch by short_code (or SSR pass-down)
  use-accept-offer.ts
  use-lead-document-upload.ts       # Presigned upload helper
```

---

11. AI Layer

11.1 General principles

- AI **MUST NOT** be a conversational chatbot.
- AI **MUST** be a structured-output service: take inputs, return JSON.
- AI **SHOULD** fail gracefully — if AI service is down or rate-limited, deterministic fallbacks return useful results.
- Use Anthropic Claude (existing infra; `anthropic` SDK or direct API).
- Cache aggressively — same inputs should not re-trigger LLM calls within 5 minutes.
- Log every call to a new `ai_call_logs` table (for cost tracking + debugging).

11.2 Functions

`ai-extract-from-conversation`

Inputs: `{ leadId, conversationId, sinceMessageId? }`

Behaviour:

- Loads recent messages.
- Prompts LLM with: schema of `application_data` fields + current values + conversation transcript.

- LLM returns: `{ field: 'years_driving', value: 3, confidence: 0.9, evidence: 'said "I have been driving for 3 years"' }`.
- Returns only updates with confidence ≥ 0.7.

UI: shown as suggestion chips in left column. Admin clicks "Accept all" to merge into `application_data`.

`ai-rank-matches`

Inputs: `{ leadId, matchOptions: MatchOption[] }`

Behaviour:

- Prompts LLM with lead profile + each option.
- Returns per option: `{ optionIndex, aiScore (0–100), acceptanceProbability (0–1), reasoning: string }`.

Cache key: `leadId + hash(matchOptions)`.

`ai-suggest-next-action`

Inputs: `{ leadId }`

Behaviour:

- Loads lead state, recent messages, stage, time-in-stage.
- Prompts LLM with stage-specific playbook.
- Returns: `{ action: 'send_doc_request' | 'send_followup' | 'send_offer' | 'mark_lost' | 'do_nothing', confidence, draftMessage? }`.

If `confidence < 0.6`, returns `do_nothing` (don't surface a noisy suggestion).

`ai-draft-message`

Inputs: `{ leadId, intent: 'welcome' | 'doc_request' | 'approval' | 'offer' | 'followup' | 'decline' | 'custom', customPrompt? }`

Behaviour:

- Loads tenant tone settings (a new field: `tenants.communication_tone` — `casual` / `professional` / `friendly`, default `friendly`).
- Drafts a message in that tone with appropriate variables.
- Returns: `{ subject?, body, channelHint: 'sms' | 'email' | 'whatsapp' }`.

11.3 Cost & rate limits

- Add tenant-level monthly AI quota (`tenants.ai_monthly_quota` default 1000 calls).
- Hard stop at quota with admin-visible warning.
- Free quota for V1 — billing for AI usage is a separate future project.

---

12. Reuse Map

This feature heavily reuses existing Drive247 infrastructure. The table below is **the source of truth** — do not re-implement anything listed in the right column.

Lead Hub need	Existing Drive247 surface	File reference
Lead capture (quick enquiry path)	`submit-enquiry` edge function	`supabase/functions/submit-enquiry/index.ts` (extend to write to `leads`)
Existing enquiry page UI patterns	`/enquiries` page	`apps/portal/src/app/(dashboard)/enquiries/page.tsx`
Stat-card + filter-bar + drawer pattern	Existing enquiry detail	`apps/portal/src/components/enquiries/enquiry-detail-drawer.tsx`
Manager permissions tab keys	Existing system	`apps/portal/src/lib/permissions.ts` — add `leads` + `automations`
Multi-tenant RLS helpers	`get_user_tenant_id()`, `is_super_admin()`	Postgres helpers — already exist
Customer model & profile	`customers` table	DB schema
Identity verification	Veriff / AI / CMD	`supabase/functions/create-veriff-session/`, `create-ai-verification-session/`, `ai-document-ocr/`, `ai-face-match/`, CMD verification
Insurance qualification	Bonzah	`supabase/functions/bonzah-*`
Vehicle availability + calendar	Existing booking calendar + `external_bookings` (Turo/Airbnb sync)	DB schema + booking app
Pricing engine	Daily/weekly/monthly tiers + dynamic pricing	Existing booking pricing logic
Min rental hours/days	Existing	`tenants.min_rental_hours`, `tenants.min_rental_days`
SMS infra	Twilio / AWS SNS	`supabase/functions/aws-sns-sms/`, Twilio sandbox + 10DLC config
Email infra	SES / Resend	`supabase/functions/aws-ses-email/`, `_shared/resend-service.ts`
WhatsApp infra	Twilio WhatsApp	`supabase/functions/send-collection-whatsapp/`, `send-signing-whatsapp/`
Email templates with variables	Existing template service	`apps/portal/src/components/lockbox-templates/*` pattern
Cron-based polling	`cmd-poll-pending` + `reminder_config`	`supabase/functions/cmd-poll-pending/` (just added)

Lead Hub need	Existing Drive247 surface	File reference
Realtime channels pattern	`RealtimeChatContext`	`apps/portal/src/contexts/RealtimeChatContext.tsx`
Customer chat	`customer-chat` edge function	`supabase/functions/customer-chat/`
BoldSign e-sign send	Existing helper	`supabase/functions/_shared/boldsign-client.ts`
Stripe Connect deposit holds	`create-preauth-checkout`	`supabase/functions/create-preauth-checkout/`
Stripe Checkout for payment links	`create-checkout-session`	`supabase/functions/create-checkout-session/`
Tenant feature flags pattern	`enquiries_enabled` , etc.	`tenants` table
Migration naming convention	`YYYYMMDDHHMMSS_description.sql`	`supabase/migrations/`
Storage bucket policies	`gig-driver-images` pattern	`storage.buckets` + policies
`useTenant()` hook	`TenantContext`	`apps/portal/src/contexts/TenantContext.tsx`
React Query setup	Existing per-app `QueryClientProvider`	App layouts
Toast notifications	`@/hooks/use-toast` (portal), `sonner` (booking)	Existing
Form patterns	React Hook Form + Zod	Existing schemas
Tab keys + UI for manager permissions	`manager-permissions-selector.tsx`	`apps/portal/src/components/users/`
Drag-drop UI library	Use `@dnd-kit/core` (new, but standard)	New install
Flow builder UI library	Use `@xyflow/react` (React Flow)	New install

13. Manager Permissions

Extend `apps/portal/src/lib/permissions.ts` with:

```
export const TAB_KEYS = {
  // … existing
  LEADS: 'leads',
  AUTOMATIONS: 'automations',
} as const;

export const TAB_GROUPS = {
  // … existing
  PIPELINE: {
    label: 'Pipeline',
    tabs: ['leads', 'automations'],
  },
};

export const ROUTE_TAB_MAP = {
  // … existing
  '/leads': 'leads',
  '/automations': 'automations',
};
```

Granularity

- `leads` (viewer | editor)
  - viewer: read board, read workspace, read messages
  - editor: all the above + create/edit/delete leads + send messages + stage changes + create offers + convert
- `automations` (viewer | editor)
  - viewer: read list, read builder
  - editor: edit drafts, **but publishing requires `admin` or `head_admin` role** — enforced in `automation-publish` edge function

UI gating

Sidebar items, board access, and quick-action buttons filtered via `useManagerPermissions().canView('leads')` / `canEdit('leads')`.

---

14. Tenant Feature Flags

14.1 New flags on `tenants`

```
ALTER TABLE public.tenants
  ADD COLUMN lead_management_enabled BOOLEAN NOT NULL DEFAULT false,
  ADD COLUMN automations_enabled BOOLEAN NOT NULL DEFAULT false,
  ADD COLUMN lead_stale_threshold_hours INT NOT NULL DEFAULT 48,
  ADD COLUMN lead_auto_lost_threshold_hours INT NOT NULL DEFAULT 168,
  ADD COLUMN communication_tone TEXT NOT NULL DEFAULT 'friendly';  -- casual | friendly | professional
```

14.2 Settings UI

New settings tab: **Lead Management**

Route: `apps/portal/src/app/(dashboard)/settings/lead-management/page.tsx`

Fields:

- Toggle: Lead Management enabled
- Toggle: Automations enabled (requires Lead Management enabled)
- Stale lead threshold (hours)
- Auto-lost threshold (hours)
- Communication tone (radio)
- Application form public URL (display only) — `https://{slug}.drive-247.com/apply`
- Default message templates (link to template editor)

Gating:

- Toggling `lead_management_enabled=true` adds `/leads` to the sidebar and exposes `/apply` on the booking site.
- Toggling `automations_enabled=true` adds `/automations` to the sidebar.
- **MUST** be reflected in `app-sidebar.tsx` via `useTenant().tenant.lead_management_enabled`.

---

15. Templates & Notifications

15.1 System default templates (seeded on tenant enable)

When a tenant flips `lead_management_enabled=true`, seed default rows in `lead_message_templates`:

Channel	Category	Name	Body (sketch)
sms	welcome	Default Welcome	"Hi {{first_name}}, thanks for applying with {{tenant_name}}. We'll be in touch shortly."
sms	doc_request	Default Doc Request	"Hey {{first_name}}, please upload your licence and a selfie here: {{doc_upload_link}}"
sms	approval	Default Approval	"{{first_name}}, you're approved! Here's your vehicle offer: {{offer_link}}"
sms	offer	Default Offer	"{{first_name}} 👋 I picked some cars for you for {{start_date}} – {{end_date}}: {{offer_link}}"
sms	reminder	Default Reminder	"Hi {{first_name}}, just checking in on your rental application. Let me know if you have any questions."
sms	decline	Default Polite Decline	"Hi {{first_name}}, unfortunately we won't be able to rent to you at this time. Best of luck."
email	…	(mirror set with subject + body)	
whatsapp	…	(mirror set)	

Seeded via a Postgres function `seed_default_lead_templates(tenant_id)` called from the settings toggle handler.

15.2 Variable catalog

Available `{{variables}}` in all templates:

```
{{first_name}}            → leads.full_name first token
{{full_name}}
{{phone}}
{{email}}
{{vehicle}}               → name of vehicle (offer-context only)
{{start_date}}
{{end_date}}
{{rental_length}}
{{weekly_rate}}
{{total_price}}
{{offer_link}}            → full URL to /offer/[code] (offer-context)
{{doc_upload_link}}       → secure upload URL
{{agreement_link}}        → BoldSign signing URL
{{deposit_link}}          → Stripe checkout URL
{{pickup_link}}           → pickup scheduler URL
{{tenant_name}}
{{tenant_phone}}
{{tenant_email}}
{{operator_first_name}}   → assigned staff first name (if assigned)
```

Rendered server-side in `send-lead-message` via the same engine as lockbox-templates.

15.3 Notification routing

For each outbound message:

1. Determine channel (operator choice or automation step).
2. Render variables.
3. Insert `conversation_messages` row with `direction='outbound'`, `status='queued'`.
4. Dispatch via channel:
   - SMS → existing `aws-sns-sms` (or Twilio if tenant is on Twilio 10DLC)
   - Email → existing `aws-ses-email` or `resend-service`
   - WhatsApp → Twilio WhatsApp via existing `send-collection-whatsapp` infrastructure
5. Capture provider response → update message row (`status='sent'`, `channel_message_id`).
6. Provider delivery webhook updates `status` to `delivered` / `read` / `failed`.

15.4 Inbound routing

Twilio + SES webhooks → existing infrastructure routes inbound messages:

- Look up `phone_normalised` / `email_lower` → find lead → append to its conversation.
- If no match → create a new lead with `source='inbound_*'`.

---

16. Realtime Channels

16.1 Subscriptions

Surface	Subscription
Kanban board	`leads` table filtered by `tenant_id`
3-column workspace	`leads.id=X` + `conversation_messages.conversation_id=Y` + `automation_runs.entity_id=X` + `lead_activity.lead_id=X`
Automations list	`automations.tenant_id=X`
Offer-link analytics	`lead_offers.lead_id=X`

16.2 Hook pattern

```
function useRealtimeLeads(tenantId: string) {
  const queryClient = useQueryClient();

  useEffect(() => {
    if (!tenantId) return;
    const ch = supabase.channel(`tenant_${tenantId}_leads`)
      .on('postgres_changes', {
        event: '*',
        schema: 'public',
        table: 'leads',
        filter: `tenant_id=eq.${tenantId}`,
      }, () => {
        queryClient.invalidateQueries({ queryKey: ['leads', tenantId] });
      })
      .subscribe();

    return () => { supabase.removeChannel(ch); };
  }, [tenantId, queryClient]);
}
```

Mirror the pattern from `RealtimeChatContext`.

16.3 Presence

For 3-column workspace, broadcast presence so staff know if a teammate is viewing the same lead:

```
Channel: `lead_${leadId}_presence`
Payload: { user_id, name, avatar_url, joined_at }
```

Show small avatar dots in the top-right of the workspace.

---

17. Phasing & Build Order

Phase 1 — Lead Hub MVP (the priority)

Sequence (each task assumes the previous is done):

1. **Migrations** (Section 8.1) — all new tables, indexes, RLS, triggers, realtime publications.
2. **Tenant flags + settings page** (Section 14).
3. **Backend: lead capture pipeline** — `submit-application`, `check-blacklist-match`, `compute-lead-score`, `submit-quick-enquiry` refactor.
4. **Backend: data hooks** — refactor existing `submit-enquiry` to write to `leads`; migrate `enquiries` data.
5. **Booking app: Apply form** — multi-step wizard + submitted page.
6. **Portal app: Leads index page** — table view first (re-skin `/enquiries`), then Kanban.
7. **Portal app: 3-column workspace** — left (info/docs/notes/activity) + middle (conversation).
8. **Conversation infrastructure** — `conversations` + `conversation_messages` + `send-lead-message` + inbound webhooks.
9. **Right column: Matching engine** — `run-matching-engine` + UI.
0. **Right column: Offer-link builder** — `create-offer-link`, `view-offer`, `accept-offer`, customer offer page.
1. **Right column: Quick actions** — wire to existing Veriff/Bonzah/BoldSign/Stripe.
2. **Convert to Rental** — `convert-lead-to-rental` + dialog.
3. **Blacklist UI** — add to blacklist, blacklist tab, polite decline template.
4. **Hardcoded automations** (not yet builder):
   - Welcome SMS on `lead.created`
   - Stale-lead reminder at 24h, 48h
   - Auto-lost at 7d
   - Offer expiry → lead.offer_expired event Hardcode these in the relevant edge functions / cron tasks.
5. **Realtime everywhere** — invalidate queries on lead/message changes.
6. **AI v1** — `ai-suggest-next-action` only. Other AI functions deferred to Phase 2.
7. **Testing + polish** — acceptance criteria (Section 18).
8. **Tenant onboarding** — settings UI to enable + default template seeding.

Phase 2 — Automations Module

1. **Migrations** — `automations`, `automation_steps`, `automation_runs`, `automation_run_logs`, `automation_event_queue`.
2. **Event registry shared lib** (TS + Deno).
3. **Engine** — `automation-trigger-event`, `automation-execute-step`, `automation-poll-pending` (cron).
4. **Sidebar nav** — `/automations`.
5. **Builder UI** — list page + React Flow canvas + node types + properties panel.
6. **Publish flow** — snapshot + version bump.
7. **Test mode**.
8. **Attach to lead** dropdown in 3-column workspace.
9. **Migrate hardcoded automations** from Phase 1 into shipped default automations (cloneable).
0. **Phase 2 actions**: WhatsApp, move-stage, assign-staff, create-task (optional).

Phase 3 — AI rerank + extraction

1. `ai-rank-matches`
2. `ai-extract-from-conversation`
3. `ai-draft-message`
4. AI suggestion chips integrated throughout

Phase 4 — Operator-configurable application form builder

Not in scope here. Design separately.

---

18. Acceptance Criteria

Use these for QA. Each is a GIVEN/WHEN/THEN.

18.1 Customer Apply flow

- **GIVEN** tenant with `lead_management_enabled=true` **WHEN** customer completes the 7-step Apply form and submits **THEN** lead row is created in `leads` with `source='application'`, `stage='new'`, application_data populated.
- **GIVEN** Apply submission with phone matching an existing blacklist entry **WHEN** customer submits **THEN** lead is created with `stage='blacklisted'`, `blacklist_match_id` set, no SMS sent.
- **GIVEN** Apply submission with phone matching an existing non-terminal lead **WHEN** customer submits **THEN** no new lead is created; existing lead's `application_data.submissions[]` grows by one; existing lead surfaces at top of board.
- **GIVEN** tenant with `lead_management_enabled=false` **WHEN** customer hits `/apply` **THEN** page returns 404 / disabled state.

18.2 Kanban board

- **GIVEN** 50 leads across stages **WHEN** operator opens `/leads` **THEN** all leads render grouped by stage; columns are scrollable; counts visible.
- **GIVEN** operator drags a card from `new` to `approved` **WHEN** transition is invalid (e.g. skips `docs_verified`) **THEN** drop is rejected with a toast.
- **GIVEN** another staff member moves a card from `new` to `contacted` in another browser **WHEN** the operator is viewing the board **THEN** the card moves columns within 2 seconds (realtime).

18.3 3-column workspace

- **GIVEN** operator opens a lead's workspace **WHEN** the page loads **THEN** info, conversation, and AI/matching panel all render within 1s.
- **GIVEN** operator types a message in the composer **WHEN** they select SMS channel and click Send **THEN** message appears immediately in the thread as outbound; SMS is dispatched via Twilio; status updates to `sent` / `delivered`.
- **GIVEN** customer replies via SMS **WHEN** Twilio inbound webhook fires **THEN** within 5s the thread shows the new inbound message and the workspace badge updates.

18.4 Matching engine

- **GIVEN** a lead requesting a Corolla 25 May–5 Jun where both Corollas are booked **WHEN** the matching engine runs **THEN** result includes:
  - The requested Corolla(s) with `available='unavailable'`
  - At least one alternative in the same class with `available='full'`
  - A stitched option if a 2-vehicle combo covers the period

- **GIVEN** matching engine returns 5 deterministic options **WHEN** AI rerank is enabled and operational **THEN** options carry `aiScore` and `acceptanceProbability` fields; final order weights both.
- **GIVEN** AI rerank fails or times out **WHEN** matching engine runs **THEN** deterministic order is preserved; no error surfaced to UI.

18.5 Offer link

- **GIVEN** operator picks 3 vehicles + clicks Create offer link with SMS dispatch **WHEN** they submit **THEN** `lead_offers` row is created, lead stage moves to `vehicle_offered`, SMS is sent with `{{offer_link}}` rendered.
- **GIVEN** customer opens offer link **WHEN** the page loads **THEN** `view_count` increments, `first_viewed_at` is set on first view, `lead.offer_opened` event is emitted, 3 vehicles render with correct prices.
- **GIVEN** customer picks Civic on an offer link **WHEN** they confirm **THEN** lead stage moves to `offer_accepted`, `accepted_vehicle_id` / `accepted_*_date` populated.
- **GIVEN** offer link past `expires_at` **WHEN** customer opens it **THEN** expired screen renders; `lead.offer_expired` event emitted (idempotent — only fires once).

18.6 Blacklist

- **GIVEN** operator clicks Add to Blacklist with a reason **WHEN** confirmed **THEN** `blacklist_entries` row created with the lead's phone/email/licence, lead stage moves to `blacklisted`, polite decline SMS sent.

18.7 Conversion

- **GIVEN** lead in stage `deposit_paid` with all data populated **WHEN** operator clicks Convert to Rental **THEN** `customers` row created, `rentals` row created, `lead.customer_id` and `lead.converted_to_rental_id` set, lead stage moves to `converted`.
- **GIVEN** conversion completes **WHEN** operator opens the customer's Messages **THEN** the same conversation appears with full history from the lead phase.

18.8 Automations

- **GIVEN** a published automation with trigger `lead.created` + SMS step **WHEN** a new lead is created **THEN** within 60s the automation run is created and the SMS is sent.
- **GIVEN** automation with a 2-hour wait step **WHEN** the step is hit **THEN** run status is `waiting` with `resume_at` ~2h ahead; at resume time the next step executes.
- **GIVEN** operator edits a published automation **WHEN** they save changes **THEN** automation goes back to `draft`; existing in-flight runs continue on the previously published snapshot.
- **GIVEN** automation is published v1 then republished as v2 **WHEN** a new event triggers **THEN** new run starts on v2; existing runs stay on v1.
- **GIVEN** automation has SMS + Wait + Condition + Stop steps **WHEN** test mode runs against a real lead **THEN** preview shows each step with rendered content, without sending any actual SMS.

---

19. Anti-Patterns (What NOT to do)

These are explicit prohibitions, not preferences.

1. **MUST NOT** copy Rental Pal Pro's PDF insurance card editing workflow. It's a legal landmine. Drive247 stores official documents only; no "edit and re-issue" flows.

2. **MUST NOT** split lead and customer conversations into two threads. One conversation, two indexed identifiers (`lead_id` + `customer_id`).
3. **MUST NOT** introduce a parallel pipeline next to the existing `enquiries` flow. Refactor `enquiries` writes to go to `leads` ; deprecate the table.
4. **MUST NOT** lock vehicles when offers are created. Re-check availability only at acceptance time.
5. **MUST NOT** allow editing a published automation in place. Edits create a new draft; published snapshot stays frozen until next Publish.
6. **MUST NOT** use `pg_cron` for the automation poller if Supabase scheduled edge functions are simpler — but **must** ensure the cron runs at least every 60s.
7. **MUST NOT** call AI inline on every UI render. Cache for ≥5 min; lazy-load suggestions.
8. **MUST NOT** introduce new RLS helper functions. Reuse `get_user_tenant_id()` and `is_super_admin()` .
9. **MUST NOT** create new payment processing logic. Reuse `create-checkout-session` and `create-preauth-checkout` for deposits.
0. **MUST NOT** create new identity verification flows. Reuse Veriff / AI-OCR / face-match / CMD.
1. **MUST NOT** use raw `supabase.auth.user_id` for FK; use `app_users.auth_user_id` pattern.
2. **MUST NOT** generate documentation files unless requested.
3. **MUST NOT** add new env vars for AI without documenting in `.env.example` .
4. **MUST NOT** build a generic chat AI. AI here is extractive + suggestive only.
5. **MUST NOT** allow a non- `admin` / non- `head_admin` user to publish an automation. Validate role in `automation-publish` .
6. **MUST NOT** treat `enquiries.status='resolved'` as the same as `leads.stage='resolved'` . There is no `resolved` lead stage; map legacy `resolved` → `lost` during migration.
7. **MUST NOT** assume cross-tenant data access. Every query MUST be tenant-scoped.

---

20. Open Questions / Future Phases

Not in scope for V1/V2. Listed for future planning.

1. **Operator-configurable application form builder** (V3)
2. **AI receptionist** — full phone call AI (V3, hookup to existing call-recording feature)
3. **Cross-tenant blacklist** — opt-in shared blacklist across tenants (V3, requires legal review)
4. **WhatsApp business templates** at scale (V2 stretch)
5. **Lead scoring training** — feed historical conversion outcomes back into the score model (V3)
6. **Mobile app for operators** to triage leads (V3)
7. **Lead → multiple-rental conversion** — single lead becomes a multi-vehicle account (V3)
8. **Refer-a-friend tracking** on leads (V3)
9. **Advanced waitlist matching cron** — fire on every vehicle return event (V2)
0. **Round-robin staff assignment** automation action (V2)

---

21. Implementation Checklist (for the dev's AI)

A linear todo list to follow.

Stage 1 — Foundations

- [ ] Create migration `20260601100000_create_blacklist_entries.sql`
- [ ] Create migration `20260601101000_create_leads.sql`
- [ ] Create migration `20260601102000_create_lead_notes_documents_activity.sql`
- [ ] Create migration `20260601103000_create_conversations.sql`
- [ ] Create migration `20260601104000_create_lead_offers.sql`
- [ ] Create migration `20260601107000_tenant_feature_flags.sql`
- [ ] Create migration `20260601108000_migrate_enquiries_to_leads.sql`
- [ ] Create storage bucket `lead-documents` with policies
- [ ] Add `leads`, `conversations`, `conversation_messages`, `lead_offers` to `supabase_realtime` publication
- [ ] Add tab keys `leads`, `automations` to `apps/portal/src/lib/permissions.ts`
- [ ] Regenerate types: `npx supabase gen types typescript ... > apps/portal/src/integrations/supabase/types.ts`
- [ ] Copy types to all 3 apps (portal/booking/admin)

Stage 2 — Lead capture

- [ ] Create `apps/booking/src/client-schemas/apply.ts` (Section 6.2)
- [ ] Create `apps/booking/src/app/apply/page.tsx` + components in `components/apply/`
- [ ] Create `apps/booking/src/app/apply/submitted/page.tsx`
- [ ] Create `supabase/functions/submit-application/index.ts`
- [ ] Create `supabase/functions/check-blacklist-match/index.ts`
- [ ] Create `supabase/functions/compute-lead-score/index.ts`
- [ ] Refactor existing `supabase/functions/submit-enquiry/index.ts` to write to `leads`

Stage 3 — Portal Kanban

- [ ] Create `apps/portal/src/lib/lead-stage-machine.ts`
- [ ] Create hooks: `use-leads.ts`, `use-lead.ts`, `use-lead-mutations.ts`, `use-lead-board.ts`
- [ ] Create `apps/portal/src/app/(dashboard)/leads/page.tsx` (board)
- [ ] Create components: `lead-board.tsx`, `lead-board-column.tsx`, `lead-card.tsx`
- [ ] Install `@dnd-kit/core` for drag-drop
- [ ] Add `/leads` to sidebar with `lead_management_enabled` gate
- [ ] Replace `/enquiries` page to redirect to `/leads`

Stage 4 — Workspace

- [ ] Create `apps/portal/src/app/(dashboard)/leads/[id]/page.tsx`
- [ ] Create `lead-workspace.tsx`, `lead-info-panel.tsx`, `lead-documents-list.tsx`, `lead-notes-list.tsx`, `lead-activity-timeline.tsx`
- [ ] Create `lead-communication-panel.tsx`, `lead-message-bubble.tsx`, `lead-composer.tsx`, `lead-template-picker.tsx`
- [ ] Create hooks: `use-conversation.ts`, `use-conversation-messages.ts`, `use-send-message.ts`
- [ ] Create edge function `send-lead-message`
- [ ] Create edge functions `inbound-sms-webhook`, `inbound-email-webhook`
- [ ] Configure Twilio + SES inbound webhooks to point at these endpoints

Stage 5 — Right column (matching + offer)

- [ ] Create `supabase/functions/_shared/matching.ts`
- [ ] Create `supabase/functions/run-matching-engine/index.ts`
- [ ] Create hook `use-matching-engine.ts`
- [ ] Create component `lead-matching-engine.tsx`
- [ ] Create `supabase/functions/create-offer-link/index.ts`
- [ ] Create `supabase/functions/view-offer/index.ts`
- [ ] Create `supabase/functions/accept-offer/index.ts`
- [ ] Create `apps/booking/src/app/offer/[code]/page.tsx`
- [ ] Create offer components in `apps/booking/src/components/offer/`
- [ ] Create `offer-builder-dialog.tsx` in portal
- [ ] Create `lead-automations-panel.tsx` (Phase 1: shows Quick Actions only)

Stage 6 — Quick Actions

- [ ] Wire Request Documents button → existing Veriff/AI verification
- [ ] Wire Run Veriff button → `create-veriff-session`
- [ ] Wire Check Bonzah button → `bonzah-create-quote`
- [ ] Wire Send Agreement button → BoldSign send via existing helper
- [ ] Wire Send Payment Link → `create-preauth-checkout` / `create-checkout-session`
- [ ] Wire Schedule Pickup → existing scheduler infra
- [ ] Create `convert-lead-to-rental` edge function
- [ ] Create `convert-to-rental-dialog.tsx`
- [ ] Create `blacklist-confirm-dialog.tsx`

Stage 7 — Hardcoded automations (Phase 1 substitute)

- [ ] In `submit-application` : send welcome SMS using default template
- [ ] In `automation-poll-pending` (initial version): scan leads for staleness, send reminders, auto-lost at 7d
- [ ] On offer creation: schedule expiry transition

Stage 8 — Settings + flags

- [ ] Create `apps/portal/src/app/(dashboard)/settings/lead-management/page.tsx`
- [ ] Wire toggle to call admin edge function that flips `tenants.lead_management_enabled`
- [ ] Seed default templates on enable
- [ ] Update `app-sidebar.tsx` to gate `/leads` link

Stage 9 — AI v1

- [ ] Create `supabase/functions/ai-suggest-next-action/index.ts`
- [ ] Create `lead-ai-next-action.tsx`
- [ ] Wire suggestion chip in right column

Stage 10 — Acceptance + QA

- [ ] Verify all acceptance criteria (Section 18)
- [ ] Test multi-tenant isolation
- [ ] Test realtime updates

- [ ] Test mobile responsive layout
- [ ] Migrate existing `enquiries` rows
- [ ] Update CLAUDE.md if needed (operator-visible behaviour changes only)

Stage 11 — Phase 2: Automations Module

- [ ] Create migrations for `automations`, `automation_steps`, `automation_runs`, `automation_run_logs`, `automation_event_queue`
- [ ] Install `@xyflow/react`
- [ ] Create event registry shared lib (TS + Deno)
- [ ] Create `automation-trigger-event`, `automation-execute-step`, `automation-poll-pending`, `automation-publish` edge functions
- [ ] Create portal routes `/automations` + `/automations/[id]`
- [ ] Create components in `components/automations/`
- [ ] Migrate hardcoded automations from Phase 1 into the new module
- [ ] Add automation attach UI to lead workspace right column
- [ ] Test mode implementation
- [ ] Publish flow with versioning

---

22. References

- Existing enquiries — `apps/portal/src/app/(dashboard)/enquiries/page.tsx`, `apps/booking/src/components/enquiry/enquiry-modal.tsx`, `supabase/migrations/20260502120844_add_enquiries.sql`
- Lockbox templates pattern — `apps/portal/src/components/settings/` (template editor), `lockbox_templates` table
- Realtime chat — `apps/portal/src/contexts/RealtimeChatContext.tsx`, `apps/booking/src/contexts/CustomerRealtimeChatContext.tsx`
- Manager permissions — `apps/portal/src/lib/permissions.ts`, `apps/portal/src/hooks/use-manager-permissions.ts`, `apps/portal/src/components/users/manager-permissions-selector.tsx`
- Existing edge functions — `supabase/functions/_shared/cors.ts`, `_shared/boldsign-client.ts`, `_shared/stripe-client.ts`, `_shared/aws-config.ts`, `_shared/resend-service.ts`
- Storage bucket pattern — gig-driver-images bucket creation
- Cron polling pattern — `supabase/functions/cmd-poll-pending/`
- Tenant feature flag pattern — `tenants.enquiries_enabled`, `tenants.lockbox_enabled`

End of specification.